BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION

DIPLOMA THESIS

Automated Phishing Detection in Emails Using Artificial Intelligence

Supervisor
Professor PhD. Darabant Sergiu Adrian

Author
Bugnaru Tudor Eduard

2025

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ(ÎN LIMBA ENGLEZĂ)

LUCRARE DE LICENŢĂ

Detectarea Automată a Email-urilor de Tip Phishing Folosind Inteligența Artificială

Conducător științific
Prof. Dr. Darabant Sergiu Adrian

Absolvent
Bugnaru Tudor Eduard

2025

ABSTRACT

This thesis will focus on a fully automated system for phishing detection in emails, integrating both artificial intelligence and real-time browser interaction. The architecture proposes the implementation of three layers of defense against phishing: a deep learning model based on Long Short-Term Memory(LSTM) for text analysis, a pretrained transformer-based Uniform Resource Locator(URL) classification model and a static threat intelligence using VirusTotal Application Programming Interface(API). For deciding the final verdict, a prudent strategy is applied: if any of the aforementioned layers flags the email as being suspicious, then the final result shall be suspicious too.

The backend is built using Flask, integrates Gmail API and web push notifications for facilitating real-time inbox monitoring, while the data is stored in a PostgreSQL database. The frontend includes a modern Angular application split into components, and a published browser extension that overlays verdicts directly within the Gmail service. Multiple Artificial Intelligence models were benchmarked on a selected dataset, and the LSTM model proved to outperform classical Machine Learning methods in evaluation metrics.

The personal contributions of the work consist of multiple phishing detection mechanisms, a lightweight frontend plugin, a fullstack pipeline for monitoring and user interaction, and an integrated quiz module designed to raise user awareness through short phishing related questions. The system is deployable, scalable and proves phishing detection can be automated efficiently while also preserving usability and stability.

Contents

Chapter 1

Introduction

1.1 Motivations and Context

With the widespread use of technology and internet, security attacks have become increasingly common, creating opportunities for data breaches and information stealing. A highly anticipated cybersecurity attack method is phishing, especially email phishing due to its ease of usage and effectiveness. Even with the latest advancements in the field of Artificial Intelligence and the stronger detection engines developed by enterprise companies, multiple studies prove that purely technical measures are not enough: attackers continuously find innovative user-deceiving techniques. As a consequence, in addition to technical solutions, user vigilance must be actively cultivated through awareness trainings and clear prevention instructions that help individuals recognize and question suspicious content before harmful actions occur.

The motivation of the thesis is to bridge the gap between the two trends: technical solutions and user awareness. On one-side, it unveils an automated system that uses a combination of artificial intelligence and static analysis tools which will warn the user about potential phishing threats received in its email inbox through a web application or a browser plugin. On the other-side, a simple quiz-based feature that has the sole purpose of educating the user in the field of cybersecurity and various forms of phishing threats.

Therefore, the work performed in this thesis aims to build a proof of concept platform that unifies **automated detection** and **user training**. The solution's goal is not to out-perform enterprise architectures with extensive human workforce and high budgetary allocations, but to demonstrate that a single, approachable web application can analyze live email content and at the same time reinforce safe habits through short, scenario-based quizzes.

1.2 Objectives

The project presents a multilayer email phishing detection system that integrates preprocessing and Natural Language Processing (NLP) with advanced machine learning and deep learning techniques to achieve accurate classification. Additionally, the system incorporates static analysis tools accessed via Application Programming Interface (API) requests to boost detection capabilities.

As stated before, machine learning and deep learning methods will be presented together with vectorization and data-cleaning techniques, forming the backbone of the system's classification pipeline. These components were carefully selected and evaluated throughout the development of this thesis, with high attention given to their impact on accuracy and robustness in real-world phishing scenarios. Furthermore, the models were integrated into the core web application and the browser plugin to enable real time and highly precise detection capabilities, complemented by the quiz feature, whose purpose is to raise user awareness and actively reinforce security best practices.

Personal contributions

The following points summarize the core **personal and technical contributions** of the paper:

- A proof-of-concept work that encapsulates a web platform which pairs self-operating email phishing detection with an integrated quiz-based user training module.
- A published browser extension which filters the contents of an email through multiple layers of phishing detection, enabling automated and fast visual alerts. The extension is integrated with the Gmail email service.
- A lightweight NLP pipeline which encapsulates regex filtering, stop-word removal and composing TF-IDF unigrams/bigrams.
- Design and implementation of five AI models (Random Forest, Logistic Regression, Naive Bayes, Decision Tree, and LSTM) for text-based phishing email classification.
- A suggestive comparative benchmark of four classical ML models and an LSTM network on the same stratified split.

1.3 Thesis Structure

The paper is organized in a structured and coherent manner, such that each chapter is focused and easy to follow based on the outlined categories.

In the first chapter, an overview over phishing and general cybersecurity concepts is provided, with the purpose of familiarizing the reader towards the main topic that will be discussed.

Moving on, the next section is a comprehensive State of the Art, that covers recent methods and approaches performed in phishing detection, ranging from rule-based systems to machine learning and deep learning modern techniques, as well as hybrid approaches.

The third chapter emphasizes the methodology and the implementation of the proposed Artificial Intelligence models and the development that was made in their realization. This includes data preprocessing, feature extraction, token vectorization and training of various Machine Learning and Deep Learning methods, all applied to datasets of labeled emails.

The design and implementation of the application are thoroughly described in the next section. The several key areas from this chapter focus on Requirements, System Design, Implementation, Web Application, Testing and Deployment, each covering essential aspects in the creation and delivery of the final application.

Results analysis/experiments chapter focuses on the evaluation of the proposed models and system performance. It incorporates metrics such as accuracy, precision, recall, F1-score and confusion matrix, demonstrating the system's effectiveness and limitations in detecting phishing emails.

Lastly, the conclusions chapter aims to provide a concise summary of the thesis, highlighting the main achievements from it, while also mentioning future improvements and key takeaways.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used OpenAI's GPT-4o in order to improve the readability and grammar of the paper. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

Chapter 2

Phishing overview

2.1 Security threats

As stated in the introduction chapter, with the increasing popularity of the internet and software tools, potential security issues expanded as well and became a serious matter for both enterprise environments such as companies, and regular users alike. In a recent report, cybercrimes were in the top 10 risks for the next 10 years and they caused more than 9 trillion US dollars in prejudice in the year 2024 [Wor23]. Cyberattacks can be divided in categories based on their purpose and execution method. For instance, these can be classified into: **network-based**(Distributed Denial of Service, Man-in-the-Middle), **application-based**(Structured Query Language injection, Cross-Site Scripting) or **social engineering**(Phishing, Baiting, Impersonation). Artificial Intelligence (AI)

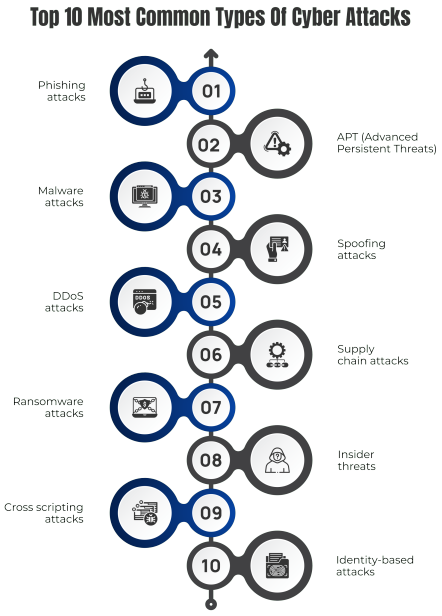


Figure 2.1: Top 10 most common cyberattacks. Source: [Sha23]

As seen in Figure 2.1, a prominent and widely spread category of cybersecurity issue are phishing attacks, which are placed first in the chart of the most common cyberattacks in 2023. The high occurrence is due to its ease of usage, since it mainly relies on exploiting human vulnerabilities such as trust, curiosity and lack of awareness. Once successfully executed, phishing rewards the attacker with the victim's sensitive information such as passwords, credit card numbers or valuable credentials.

2.2 Phishing attack life-cycle

The life cycle of a phishing attack is not very complex to be understood, even by an individual with no technical expertise. Firstly, the attacker creates a malicious copy of a legitimate website, with the purpose of gaining the credibility of the victim. Afterwards, the attacker initiates communication with a customer/user on the internet. The communication channel between the aggressor and the victim can vary from the most primitive online forms of transmissions such as email, SMS, phone call to more sophisticated approaches such as QR-code and multi-channel strategy. In this paper, the main focus will be on email phishing, one of the most commonly used channels by attackers. Subsequently, the harasser will send a well-crafted email to the user in a persuadable manner in order to convince him/her to click on a link that will be attached to the email. URLs play a significant role in the phishing life-cycle and the semantic structure of a URL can be visualized in Figure 2.2. The importance and implications of URLs will be further discussed in later chapters.

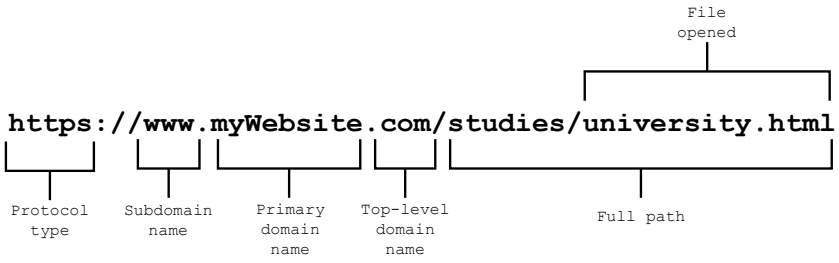


Figure 2.2: URL structure anatomy

From there on, a website will be opened that will mimic an existing original website, further strengthening the impression of legitimacy. In the opened website, the user will be prompted to input his/hers confidential information, which the attacker will use to its advantage, marking the "end" of the phishing attack [SGVS22]. Obviously, the phishing life-cycle exposed previously is only an example for the use-case of this paper, out there a dozen of other phishing cycles can be seen across different

attack scenarios (for example starting with social media profiling or using multiple steps to steal credentials and data).

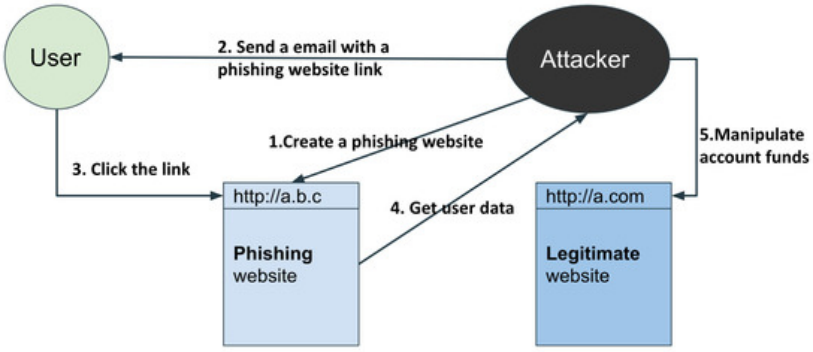


Figure 2.3: Phishing attack life cycle diagram.
[TM21]

In Figure 2.3 a graphical representation of the aforementioned phishing attack flow is presented, clearly reflecting its simplicity and the ease with which users can fall prey to such attacks.

2.3 Classification

Phishing classification can be performed based on two crucial criteria: phishing based on user credential stealing and phishing based on user information control. The first category can be divided into other 2 groups: social-engineering based and malware based phishing attacks. These further can be split into subcategories, as seen in Figure 2.4 and Figure 2.5. The focus of this paper will be concentrated around the message based methodology, as it has been shown it is one of the most widely spread approach [SpC22].

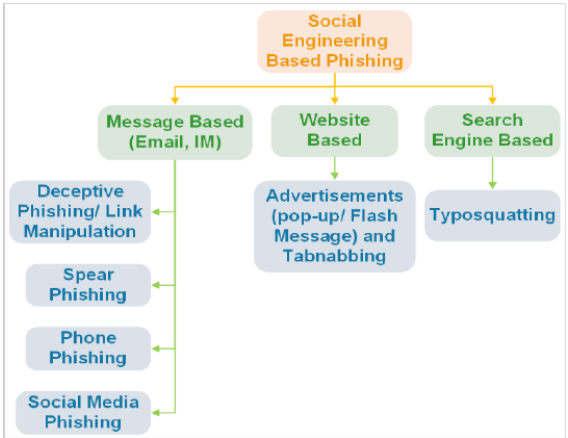


Figure 2.4: Phishing attack life cycle via email (Social Engineering)

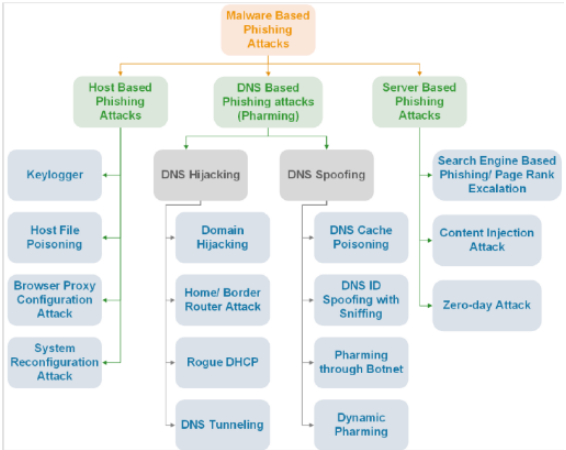


Figure 2.5: Phishing attack life cycle via email (Malware-Based)
[SpC22]

2.4 Methods of prevention

For the purpose of protecting individuals and on a bigger scale companies from phishing threats, two predominant methods have emerged: **raising human awareness** and modern **software solutions**. Each of these have a crucial role and complement each other, creating a robust defense system against evolving phishing attacks.

2.4.1 Human awareness

On one hand, companies and individuals alike invest significant resources in educating users on the importance of actively recognizing misleading messages and promoting safe online behavior. Common strategies include:

- **Security training sessions** for employees where interactive simulations of phishing attacks take place.
- **Resources and constant updates** on real-world modern phishing strategies.
- **Security policies** such as multi-factor authentication or strong password requirements.

As reviewed in [JGST20], people enhance their ability to correctly handle phishing emails upon completing anti-phishing trainings, proving their efficiency. However, as showcased in the same paper, a combination of an inadvertent individual and a well-crafted phishing message can still lead to successful phishing attacks.

2.4.2 Software tools

On the other hand, using technology as a defense tool is a complement to the first category and can prevent attacks even in the cases where the user is unaware of a potential phishing threat. Examples of software-based anti-phishing tools are:

- **Email filters** powered by well-known software companies such as Google and Microsoft.
- **URL analysis systems** which can mitigate phishing attempts before they reach the end-user.
- **Browser extensions** built-in browser warnings for potentially malicious websites.

Despite the advancement of Artificial Intelligence and its involvement into the aforementioned software and their power, these systems are not foolproof and various methods such as zero-day attacks can demonstrate their vulnerabilities.

2.5 Incident response

Incident response is a crucial aspect of cybersecurity, as it refers to the methodologies used for detecting, mitigating and responding to cyberthreats. In Figure 2.6, the fundamental process flow for incident response is visually depicted. Four vital steps can be identified: **preparation**, **detection & analysis**, **containment eradication & recovery** and **post-incident activity**.

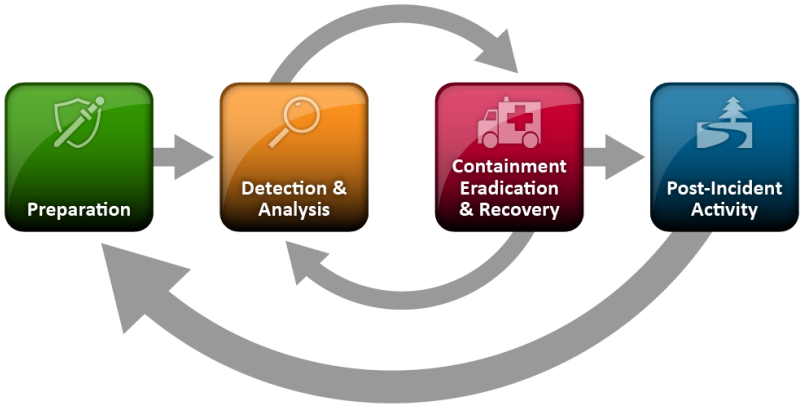


Figure 2.6: Incident response flow
[NRSS25]

Phishing attacks, similar to most other cyberattacks, follow the same methodology of incident response. They represent a potent and easy to execute type of attack: a single moment negligence or a simple click can lead to serious security issues(*credential theft*, *malware installation*, *privilege escalation*). Therefore, a proper and well-defined incident response plan is needed for companies that may eventually face such cyberthreats. [CAA+23] proposes such a plan, consisting of six steps which align to the previously fundamental incident response procedure:

- Re-provisioning suspected or confirmed compromised accounts
- Auditing account access following a confirmed phishing incident
- Isolating the affected workstation after the detection of a phishing attack
- Analyzing the malware
- Eradicating the malware
- Restore systems to normal operations and confirm they are functioning properly

A structured incident response plan is essential in mitigating phishing attacks. By following standardized steps: detection, containment, analysis, and recovery organizations can minimize damage and restore operations fast and efficiently. This thesis's focus on phishing detection directly supports such response efforts, boosting both speed and effectiveness of addressing cyberthreats.

Chapter 3

State of the Art

3.1 Text-based analysis

Phishing and concretely email phishing is considered to be a classification problem, as the quintessence of the attack detection output is to determine whether an email is legitimate or not. Binary classifying emails can be carried out based on various features as per [SGVS21]: **email body-based characteristics**, **subject-based characteristics**, **URL-based characteristics**, **script-based features** and **sender-based characteristics**. The first section, email body-based characteristics, refers to possible phishing indicators found in the email body such as urgency phrases, requests for sensitive information or reward baits. These indicators form the basis of early phishing threat detection, as they are strongly associated with fraudulent attempts. Consequently, potential victims often experience anxiety and stress, which increases their likelihood of falling for such attacks. Specific linguistic assets translate neatly into machine-readable features such as keyword flags, sentiment scores and vector embeddings. Based on this hypothesis, machine learning and natural language processing models are well-suited for training on labeled examples(phishing and legitimate) and can produce accurate classifications even in cases of zero-day attacks.

The machine learning methods that have been used recently will be described below, together with their advantages and drawbacks, starting with the supervised subcategory as seen in Figure 3.1.

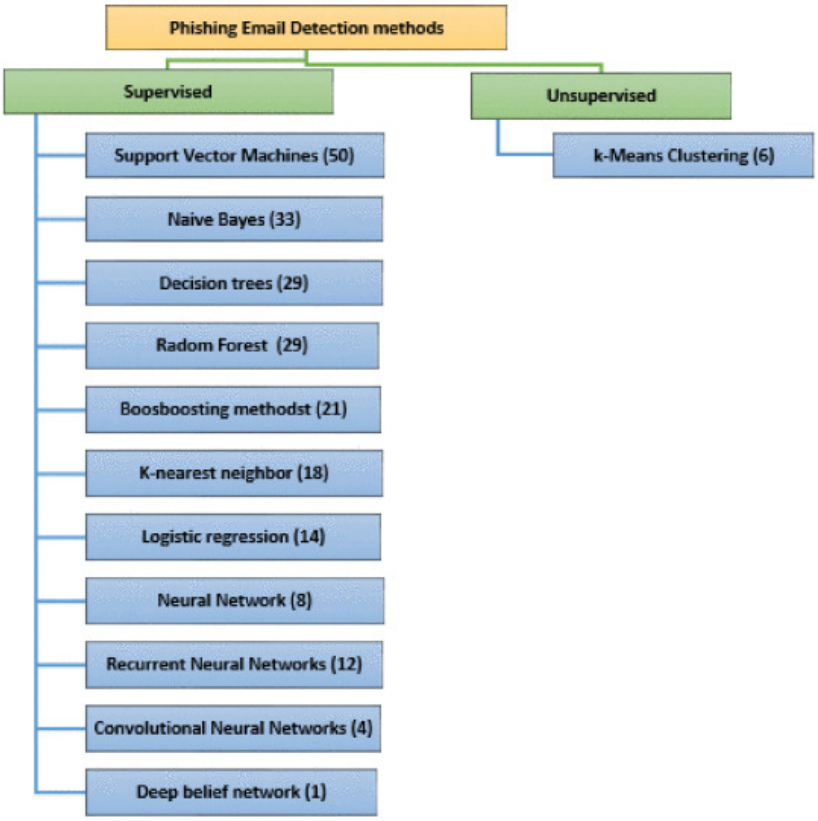


Figure 3.1: Frequency of various Artificial Intelligence techniques for email phishing detection [SGVS22]

3.1.1 Natural language processing

Natural language processing (NLP) lies at the heart of text-centric phishing detection since email content is, by definition, language. Leveraging NLP allows a model to move from simple keyword flags to capturing richer, context cues that distinguish social-engineering legitimate attempts. In this context, NLP helps enhance the previously mentioned features by converting the email body and subject into structured data that can be used by ML models.

Practically, attackers must still convince a human reader to comply, so they rely on patterns such as urgency (*verify your account within 24 h*), authority impersonation, and emotional manipulation. NLP techniques, ranging from statistical n-grams to contextual embeddings quantify these patterns and thus boost recall without using hand-crafted rules [SGVS21].

Common NLP feature pipelines. Recent surveys highlight three broad approaches [SGVS22]:

1. **Bag-of-Words / n-gram models** Term frequency–inverse document frequency (TF-IDF) vectors, possibly enriched with character n-grams, remain competi-

tive for lightweight deployments and represent strong baselines for ML algorithms.

2. **Distributed representations** Word2Vec, GloVe, or FastText embeddings project tokens into the semantic dense space, capturing synonymy ("*update*" $\approx$ "*verify*") and word similarity.

3. **Transformer-based encoders** Fine-tuned BERT-style architectures directly output a phishing/legitimate label from the [CLS] token. These models automatically focus on key elements like named entities and sentence structure, achieving top F_1 scores on benchmark datasets such as Enron-Spam and Nazario.

Limitations and outlook. NLP models can be susceptible to adversarial text perturbations such as misspellings and domain drift. Therefore, we complement them with URL and sender-based features to hedge against purely lexical evasion, mirroring recent ensembles. Future work may explore prompt-engineering LLM for few-shot adaptation, however resource constraints remain an open challenge for production systems.

3.1.2 Machine Learning methods

The linguistic features extracted from an email's subject and body must ultimately be mapped to a binary decision: *phish* or *legitimate*. ML offers a principled way to learn this decision boundary directly from labeled data avoiding hand-coded rules.

Support Vector Machines

Support Vector Machine(SVM) is an extension of the perceptron which is currently used in classification and regression problems. The method plots every data item as a point on a plane, and its goal is to find the most fitting hyperplane that separates the data into two classes. It is a fairly memory-expensive algorithm due to the use of an NxN kernel matrix, which is necessary for computing the similarity between points of data and classify them properly.

SVM has been widely used in the field of text-based phishing detection, as authors of [CSH+24] and [VGG20] attained 96.43% and 98.77% respectively accuracy scores. Sequential Minimal Optimization(SMO) is a method of speeding up SVM training by breaking the large quadratic optimization into analytically solvable two-variable subproblems with great results in the phishing classification problem [SBDD19].

Logistic Regression

Another widely used procedure for text phishing detection is Logistic Regression, which is focused on regression tasks as its name suggests. In the current context, it is used for predicting a continuous value, more specifically outputting the probability of an email as being legitimate or not based on the extracted features and other characteristics. In a study by [VGG20], the authors implemented a logistic regression classifier on their dataset, ultimately achieving a remarkable 98.56% accuracy in phishing emails classification.

Decision Trees

Decision trees can be an advantageous tool when recognizing phishing attacks due to their ease of use and simple visualization. The algorithm takes a series of decisions to determine whether an email is legitimate, and these decisions are based on several questions influenced by indexes. The most common ones are Gini Index and Entropy, where the first one reflects the probability of extracting a feature from an email that will be misclassified and the second one represents the confusion scale in comparison with the information gain. Decision trees can accomplish both regression and classification tasks.

In the study published by [VGG20], it is highlighted a decision-tree model trained on NLP-extracted features from emails achieving a 97.55% accuracy in distinguishing phishing from legitimate messages.

Random Forests

Random forests can be easily described as a combination of decision trees. These random decision tree classifiers are assigned to random sub-samples of the dataset. When training is done for each of the decision trees and the class prediction is determined, a voting takes place, and the class with the highest number of votes will be provided as output. There is a high chance of outperforming the aforementioned method(Decision Trees), due to the build-up generated by the result of multiple decision trees, albeit at the cost of increased computational complexity and slower execution time.

In the paper [AAAA25], where multiple machine learning and deep learning methods are compared and analyzed, RF manage to achieve one of the highest accuracy scores among ML algorithms, more specifically 97.23%, while the evaluation was performed on 8 merged email phishing datasets with balanced data.

Naive Bayes

The Naive Bayes(NB) classifier is a machine learning approach based on the probabilistic Bayes rule. It relies on the fact that it can predict if an event will happen or not with high accuracy based on a previous event occurrence, while also assuming that the features are conditionally independent given the target class. Briefly explained, the algorithm uses previous probability distribution of discrete events to decide the classification output of a sample. This probability can be rewritten as follows:

$$P(\text{class} \mid \text{features}) = \frac{P(\text{class}) \cdot P(\text{features} \mid \text{class})}{P(\text{features})} \tag{3.1}$$

Where:

- $P(\text{class} \mid \text{features})$ — Posterior Probability
- $P(\text{class})$ — Prior Probability of the Class
- $P(\text{features} \mid \text{class})$ — Likelihood (Probability of the features given the class)
- $P(\text{features})$ — Prior Probability of the Features (Evidence)

In both [AAAA25] and [VGG20], NB demonstrated strong performance as an efficient and highly accurate classification method, achieving 92.93% accuracy on the eight merged datasets and 98.70% on a standard dataset.

Hybrid approaches

Combining various methods of email phishing detection can improve the overall robustness and precision of the models, making them more potent in the face of evolving attack techniques. By blending together models which excel at different aspects of the problem, hybrid systems can achieve better evaluation metrics than a baseline model alone.

To reinforce this point of view, [KCD20] employed a powerful hybrid strategy that integrates NLP-driven feature extraction with both a SVM and a Probabilistic Neural Network, using an XOR-based ensemble to merge their outputs. The results were impressive; in contrast to the standalone SVM which obtained an 87% accuracy, the hybrid methodology managed to obtain a score of 98.3%, significantly outperforming the naive classifier.

Drawbacks of Machine Learning-Based Detection

While traditional machine learning algorithms such as RF, SVM, and NB have been widely applied in phishing email detection, they come with several limitations

that affect their overall effectiveness in more complex or dynamic environments.

- **Feature Engineering Dependency:** These models rely heavily on manually crafted features such as word frequency, URL length, and header field anomalies. This manual step requires expert knowledge and can lead to biased or incomplete representations of phishing behavior.
- **Limited Generalization:** Classical machine learning models often struggle to generalize well to novel or zero-day phishing attacks that differ significantly from the training data, making them less adaptable to evolving threats.
- **Data Imbalance Sensitivity:** Phishing datasets typically contain far fewer malicious samples than legitimate ones. Many classical algorithms are sensitive to such class imbalance, often resulting in high false negatives unless specific data balancing techniques are applied.
- **Scalability Concerns:** As email volume increases, maintaining performance and accuracy becomes challenging, particularly for models which do not scale efficiently with large feature sets.

As a result of these limitations, traditional machine learning approaches are increasingly supplemented, or even replaced, by more robust and adaptive Deep Learning models in modern phishing detection systems.

3.1.3 Deep Learning Methods

Deep Learning(DL) has become one of the most prominent methods for analyzing email text in modern anti-phishing systems. In comparison to traditional rule-based methods, neural networks automatically learn patterns of semantic and syntactic features that can distinguish malicious from legitimate messages. The following section will review the most relevant architectures developed in recent years, discussing their strengths and limitations.

Model Type	Input Type	Typical Use in Phishing Detection
CNN (Convolutional Neural Network)	Email text as character/word sequences	Feature extraction from email content
RNN (Recurrent Neural Network)	Sequential data (e.g., headers, text)	Detecting temporal or sequential patterns
LSTM (Long Short-Term Memory)	Long sequences of text	Capturing context in phishing message bodies
Transformer/BERT	Tokenized email content	Context-aware classification using attention
MLP (Multilayer Perceptron)	Hand-crafted features (header-based)	Basic phishing email classification

Table 3.1: Common Deep Learning Models in Phishing Detection

Table 3.1 summarizes the DL architectures most frequently adopted in recent phishing-detection literature. Each model family specializes in a different aspect of the email

aspects: Convolutional Neural Networks(CNN) excel at local lexical patterns, Recurrent Neural Networks/Long Short-Term Memory variants capture long-range context, and transformer encoders (e.g., BERT) offer holistic, attention-based representations.

Recurrent Neural Networks

RNNs are designed to model sequences of data, processing text token-by-token, carrying hidden states that capture context. Phishing emails often contain patterns in sentence structure and phrasing that evolve across the message body. Variants of RNNs models, such as LSTM mitigate vanishing gradient issues and have the capabilities of remembering long-range dependencies such as the relationship between an urgent subject line and a usage of suspicious word further down the message. Furthermore, LSTMs are particularly effective for detecting subtle cues such as urgency, fear, or impersonation phrases in the flow of text. The authors of [AA23] trained a RNN and a LSTM model on a dataset of email body texts and split 70%/30% into training and test partitions. The previously mentioned models obtained an accuracy of 98.58% and 98.89% respectively, with the LSTM model yielding a small increase in all evaluation metrics.

Convolutional Neural Networks

Although first popularized for image recognition, CNN adapt naturally to sequence data by sliding one-dimensional filters over token embeddings. The architecture used in phishing email classification can be summarized as follows:

1. **Token-embedding matrix** – The sequence of tokens is stacked into a matrix (L × d), where L is the number of tokens and d is the embedding size. 1-D convolution layer – Shared filters of width k slide vertically along the sequence axis, treating the d embedding channels like RGB channels in an image. Each filter produces a feature map that reacts to n-gram patterns such as 'verify account now'.

2. **Max-pooling** – For every feature map, global max-pooling picks the strongest activation, yielding a fixed-length vector regardless of email length.

3. **Dense layers with ReLU** – One or more fully connected layers combine pooled cues into higher-level concepts (e.g., *urgency tone + spoofed link*).

4. **Sigmoid output** – A final sigmoid (or softmax for multi-class setups) converts the logit to the probability that the message is *suspicious*.

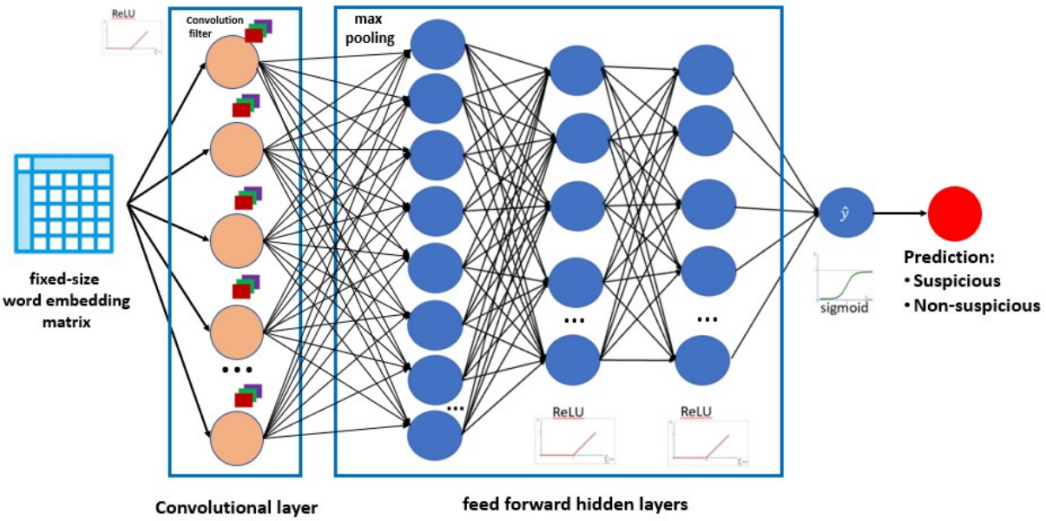


Figure 3.2: CNN feature map visualization
[AA23]

The analysis performed in [AAAA25] shows that CNNs attain a high accuracy compared to both machine learning and deep learning models across various datasets such as Enron, SpamAssasin, or TREC and even on experiments performed on the versions of multiple sets: 98.09% on merged datasets and over 97% on standalone datasets.

Transformer Models

Transformer-based models such as Bidirectional Encoder Representations from Transformers(BERT) offer state-of-the-art results in natural language understanding. When fine-tuned on phishing datasets, these models can identify phishing attempts by understanding context, identifying mischievous intents, and differentiating between genuine and fraudulent messages.

Their attention mechanisms permit them to focus on the most relevant parts of the message, such as urgent actions. In the evaluation carried out in [AAAA25], transformer-based models (BERT, DistilBERT and RoBERTa) outperformed other approaches achieving near-perfect accuracy across all 10 phishing datasets which were used for training and testing. A worth-mentioning result is that of the BERT model, that reached a perfect 100% accuracy on the Ling dataset, while also maintaining very good results on merged datasets.

Large Language Models

Recent research show that GPT-style Large Language Models(LLM) can outperform traditional ML and DL classifiers when given clear instructions or fine-tuned. In addition to their accuracy, they also provide explanations in natural language,

making their predictions easier to understand and trust. Writers of [DGV24] introduced a GPT-4o powered model named *APOLLO* which takes as prompt input raw email header and body information, returning a classification decision and also a natural language explanation of their resolution. On a benchmark, the model scored a 97% accuracy which rises to 99% after adding external threat-intel data.

Hybrid approaches

Similarly to the case of hybrid ML techniques, deep learning methods can also combine complementary neural networks. For example, CNNs can work well for capturing local patterns and RNNs for modeling sequence context. The 2024 study [AATAA24] uses raw email text preprocessed with tokenization and FastText embeddings to train a hybrid CNN and Bi-GRU model. A Bidirectional Gated Recurrent Unit(Bi-GRU) is a type of RNN variant that processes sequences in both forward and backward directions to capture contextual information from both past and future tokens. Trained with the Adam optimization algorithm and early stopping for regularization, the model achieved 99.68% accuracy and 100% precision, proving the efficiency of hybrid systems in the context of phishing detection.

Moreover, [AA23] authors fine-tune BERT to generate context-rich token embeddings and feed them into a LSTM classifier, this way combing both global semantics and sequential cues in phishing e-mails. For this hybrid model they obtained an accuracy of 99.61%, surpassing both CNN and RNN baseline models.

Limitations

Deep learning methods provide high precision, especially against previously unseen phishing variants. However, they come with some challenges:

- High computational cost for training and inference
- Requirement for large, labeled datasets
- Lower interpretability compared to simpler models or rule-based systems

Despite these challenges, deep learning continues to set the benchmark for phishing email detection and is increasingly used in production-level systems.

3.2 URL analysis

The Uniform Resource Locator(URL) plays a significant role in phishing threats, being the entry point for deceiving users to click on malicious links that lead to fake websites that have the sole purpose of stealing information or gain unauthorized

access to a user's system. From a language inspection point-of-view, an email can be classified as safe by both a software tool and a human being, but still contain a malicious URL that will harm the end user. As methods of prevention we have two main paths we can go through: **list-based** and **AI-based**.

3.2.1 List based methods of prevention

These methods focus on the methodology of checking potential dangerous URLs against prebuilt blacklists or whitelists. **Blacklists** are sets of URLs where each item is pointing to a unique previously confirmed malicious website, while **whitelists** represent the exact opposite: confirmed legitimate websites that are safe to access. The main drawback of these similar approaches are zero-day attacks, namely when dealing with newly crafted websites and URLs which have not been publicly classified as legitimate or suspicious. Another worth mentioning technique is using **content lists** where the primary objective is analyzing the actual content of a webpage such as keywords, layout and domain reputation. The most popular example using this approach is *CANTINA* proposed by [ZHC07]. The method uses term frequency-inverse document frequency(TF-IDF) for weighting the most frequent words used in a webpage and checking the results using a search engine. It got remarkable results for the time, obtaining 97% more accurate results than other modern tools, while its successor *CANTINA+* [XHRC11] enhanced the usage of heuristic strategies in decreasing the false positive rate, reaching only 0.4% [HFA24]. The disadvantage of this procedure is the network overhead which is generated by using the search engine.

3.2.2 Artificial Intelligence based methods of prevention

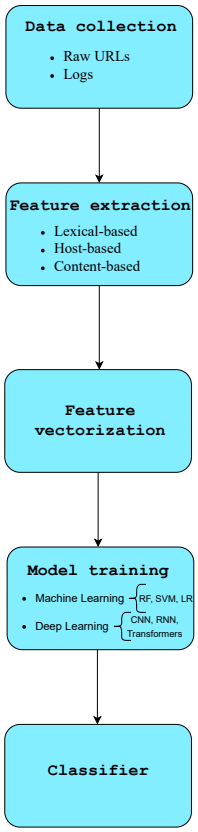


Figure 3.3: URL classification flow in Machine Learning

Artificial Intelligence based approaches to prevent phishing by detecting malicious URLs are the latest trend and have an increased popularity in research areas. In Figure 3.3, the overall flow of training a model is illustrated, highlighting its most important steps. In addition to the model training algorithms, which will be discussed in the following paragraphs, there are several essential preprocessing steps that must be completed before that.

Data collection focuses on obtaining large datasets for a larger training and testing size and thus, a more accurate model.

Feature extraction involves transforming raw URLs from datasets into meaningful patterns and indicators, allowing the models to distinguish between legitimate and phishing URLs. As seen in Figure 3.3, these can be classified as *lexical-based*

(URL length, number of hyphens), *host-based*(domain age), *content-based*(obfuscated JavaScript code).

Feature vectorization is the process of transforming raw string data into numerical values specific for each extracted feature.

Machine Learning systems rely on hand-crafted features [WKN+20], with a strong emphasis on feature extraction and feature representation. Subsequently, a binary classification is employed for output yielding, thus determining the legitimacy of an URL: *phishing* or *benign*.

The paper [ZO17] presents a lightweight URL detection mechanism that achieves 95.80% accuracy using a SVM algorithm and a limited number of six URL-based features.

In [SBDD19] the authors combine NLP features with a RF model, resulting in a powerful model with 97.89% accuracy in phishing URL classification.

Deep learning architectures for identifying harmful URLs imply less manual feature engineering and rely more on automatically learning and extracting features from raw URL strings, features such as character sequences, patterns and structures.

In the paper [and24], two DL approaches were taken, explicitly one using Artificial Neural Networks and the other using Deep Neural Networks. URL features were extracted and vectorized, and then fed into the learning systems for classification. The accuracy attained was 92% for the Artificial Neural Networks and respectively 96% for the Deep Neural Networks, proving the efficiency of DL approaches in classification problems.

The authors of [AAAM+23] developed a system of classifying URLs and compared three popular DL-based techniques namely CNN, LSTM and a combination of CNN and LSTM. The accuracies obtained were 99.2%, 97.6% and 96.8% respectively, showcasing again the very high accuracy of CNNs.

The creators of [WKN+20] achieved a 99.99% accuracy using a CNN to detect malicious URLs by learning directly from raw URL strings and processing character-level embeddings of URLs through multiple convolutional layers.

It can be observed that DL models, particularly character-level CNNs and LSTMs learn URL patterns automatically and reach approximately 99 % accuracy, eclipsing classical ML baselines.

3.3 Datasets

Much of the recent progress reported in the previous papers has been enabled by the availability of open-source corpora of phishing and benign content. Table 3.2 groups the most frequently used collections of datasets.

Dataset	Content type	Approx. size	Access	Typical use / comments
Enron	E-mail text & headers	˜35 k messages	Public	Widely adopted baseline for spam/phishing e-mail work; language is business-oriented [AAAA25].
SpamAssassin	E-mail text & headers	˜6.0 k messages	Public	Cleanly labeled ham vs. spam corpus; often merged with Enron to balance classes [AATAA24].
Ling	E-mail text	˜2.4 k messages	Public	Linguistics mailing-list mails; BERT reached **100 %** accuracy here [AAAA25].
Phishing Corpus	E-mail text	˜10 k messages	Public	Curated phishing only; paired with legitimate messages for supervised learning [AATAA24].
Kaggle Phishing E-mails (2024)	E-mail text, headers	˜70 k messages	Public	Recent multilingual set including spear-phishing samples [Sub24].
PhishTank	URL + metadata	>1 M unique URLs	Public API	Continually updated blacklist; reference ground-truth for URL models [AAAM$^+$23].
UNB ISCX URLs	URL features	87 k URLs	Public	Balanced benign/phish; includes domain-age [HFA24].
Alexa Top 1 M	URL list (benign)	1 M domains	Public	Common whitelist to complement blacklists and reduce false positives [AAAM$^+$23].

Table 3.2: Representative datasets used in recent phishing detection research.

Individual datasets capture only a slice of real-world traffic: business mail (Enron), mailing-lists (Ling) or crowd-reported URLs (PhishTank). Cross-dataset evaluation, or explicit merging as in the *Merged–10* corpus, mitigates sampling bias and reveals how well a model generalizes to unseen domains [AAAA25].

3.4 Challenges and limitations

Despite impressive headline accuracies (often >99 %), current solutions face several practical hurdles:

Data imbalance Phishing remains a low-prevalence event compared with legitimate traffic (roughly 3 of global e-mail volume [EE25]). Models trained on static, balanced corpora may under-perform once deployed, as class priors shift and criminals change their wording or hosting infrastructure. Continual fine-tuning and active learning pipelines are promising counter-measures.

Limited linguistic diversity. Most public data are using English as a language, while real attacks increasingly target users in other languages. Improving training data with multilingual samples via LLMs, or collecting region-specific corpora can improve coverage.

Adversarial adaptation. Attackers can evade text-based detectors by paraphrasing, inserting safe-looking sentences or manipulating URLs. Ensembling heterogeneous signals (NLP, HTML structure, network feedback) and adopting adversarial training are active research directions [CAA+23].

Resource constraints. Transformer-level accuracy often requires powerful Graphical Processing Units and substantial RAM amounts which is impractical for on-device filtering. Techniques like quantization, knowledge distillation, or server-based scanning can reduce resource usage, but may increase privacy risks or latency.

Overall, bridging these gaps will call for larger, more varied datasets, adaptive learning pipelines and human interpretable outputs rather than purely aiming for marginal gains in benchmark accuracy.

Chapter 4

Model Analysis and Design

4.1 Requirements

Overview

The models introduced below aim to automate the identification of phishing emails by distinguishing them from legitimate messages. The performance target is to obtain a high accuracy and precision when evaluated on an unseen test set. A probability output signifies that along with the ”phishing” or ”legitimate” label, the API returns a confidence score $\in [0, 1]$ showing how confident the model is on the prediction taken.

System environment

All trainings were carried out on a workstation equipped with an NVIDIA GeForce RTX 3050 Ti graphics card, 16 GB of RAM and an 8-core AMD Ryzen 7 5800H CPU. The most relevant software stack used are `Python` and `Tensorflow`, assisted by `NumPy`, `pandas`, `scikit-learn` for data preprocessing and evaluation, and `Matplotlib` with `Seaborn` for data visualization.

4.2 Deep Learning for text-based phishing detection

4.2.1 Model overview

The purpose of the following work is to detect phishing attacks using a DL model that only looks at the plain text of an email, specifically the subject and body. It does not rely on extra information such as headers, links, or attachments. Instead, the LSTM model used processes each email as a sequence of words and learns to spot patterns in the language that help tell the difference between phishing and

legitimate messages. LSTMs are well-suited for this type of tasks due to their ability to capture context and word order in sequential data, motivating the choice taken to use it.

4.2.2 Text Preprocessing

Raw e-mail text is first normalized to a consistent, model-accceptable form. All characters are converted to lower case so that "Example" and "example" map to the same token, thus reducing redundancy in the vocabulary and helping the model generalize better. HTML stripping then removes tags occasionally embedded by spam kits. Non-alphabetic symbols including digits and punctuation signs are discarded to shrink the vocabulary and avoid sparsity, after which any surplus of whitespace is collapsed into single spaces so that token boundaries remain clear.

The cleaned text is passed to Keras's Tokenizer, which constructs a 20000-word vocabulary from the training data. Any word that falls outside this vocabulary at inference time is replaced by a special Out-Of-Vocabulary(OOV) token, which helps the model handle new or unfamiliar terms. Consequently, each email will be represented as an integer sequence. Because recurrent networks expect equal length inputs, sequences longer than 250 tokens are truncated from the tail while shorter ones are padded with zeros on the right. This window covers more than 95% of messages in the dataset and also preserves word order.

The trained tokenizer is saved as JSON along with the model, so that new emails are processed in exactly the same way as during training.

Algorithm 1 Deep Learning model preprocess email algorithm

Require: Raw email as input string (subject + body)
Ensure: Fixed-length integer sequence representing the email

1: Convert all characters to lower case
2: Remove HTML tags using regex `<.*?>`
3: Remove non-alphabetic characters using regex `[^a-z\s]` → space
4: Normalize white-space using regex `\s+` → single space
5: Tokenize cleaned text into words using whitespace delimiter
6: Map tokens to integers using trained Keras Tokenizer
7: Replace unknown words with dedicated `<OOV>` token ID
8: **if** sequence length > MAXLEN **then**
9: Truncate sequence to MAXLEN
10: **else**
11: Right-pad sequence with `<PAD>` ID (0) to MAXLEN
12: **end if**
13: **return** Integer sequence

While many of the individual preprocessing steps follow common practices in NLP, the design, integration, and application of this specific sequence to phishing

email inputs represent an original contribution of this work.

4.2.3 Model Architecture

The classifier follows a refined sequence-encoding design. An embedding layer is at the input, transforming each vocabulary index into a dense 128-dimensional vector whose weights are learned together with the rest of the network; this allows semantically similar words (e.g., "account" and "profile") to occupy neighbouring areas in the vector space. Two LSTM layers process the sequence: the first, with 128 units, outputs a hidden state for every time-step so that the second, with 64 units, can integrate larger contextual cues. The dropout rate that which is set at 30% after each LSTM layer, has the purpose of preventing the model from depending too much on specific patterns and improve its ability to generalize.

After the LSTM layers, a dense layer with 32 Rectified Linear Unit(ReLU) activated neurons helps refine the previously learned features. Further, the sigmoid unit outputs the probability that the email is phishing. The model uses a fixed vocabulary of 20.000 words and has about 2.5 million trainable parameters - small enough to make accurate and fast prediction on a modern CPU, yet powerful to learn complex patterns as it will be demonstrated in the metric evaluation analysis section of the paper.

4.2.4 Training Strategy

Prior to optimization, the data is stratified into 80% training, 10% validation and 10% test partitions so that the class ratio remains constant across splits. The neural network is trained with the Adam optimizer, initialized at a learning rate of 1×10^{-3} and fed mini-batches of 32 padded sequences. Binary-cross-entropy serves as the loss function, aligning naturally with the probabilistic sigmoid output. This configuration of hyperparameters proved to yield the best overall performance during experimentation, making it well-suited for the binary classification task.

During training, the validation loss is evaluated after each epoch. If it fails to improve during five consecutive epochs, early stopping halts the training and restores the parameter weights that achieved the best validation performance, thus preventing overfitting as much as possible. All random seeds (`NumPy`, `TensorFlow` and `Python`) are fixed at 42, and TensorFlow's deterministic kernels are enabled to guarantee reproducibility across hardware.

A single training session on the workstation finishes in about 30 minutes. The resulting model weights (in `h5` format) and tokenizer (in `json` format) are committed then to the project's backend integrations so that downstream integration tests and deployment pipelines.

4.3 Machine Learning for text-based phishing detection

This section details the machine learning approach that serves as a reference point for the deep learning models and experiments presented later.

4.3.1 Text Preprocessing

The raw corpus comprises thousands of messages, each annotated as *Phishing Email* or *Safe Email*. To allow these messages to be fed into a ML algorithm, all these must pass through the cleaning routine shown in Algorithm 2, which performs:

- lower-casing all words;
- URL and digit stripping via regex;
- punctuation removal
- Tokenization and stop-word removal are done using NLTK (Natural Language Toolkit), a widely used Python library for processing human language data.
- re-joining the tokens into a single whitespace separated sentence for future vectorization.

Algorithm 2 Machine Learning models preprocess email algorithm

function CLEAN(t)
    $t \leftarrow$ LOWERCASE(t)
    $t \leftarrow$ REMOVEURLSANDDIGITS(t)
    $t \leftarrow$ REMOVEPUNCTUATION(t)
    $\vec{w} \leftarrow$ WORDTOKENIZE(t)
    $\vec{w} \leftarrow \vec{w} \setminus \{\text{stopwords}\}$
    return JOIN($\vec{w}$)
end function

No morphological normalization was applied - empirical tests proved that lemmatization was computationally expensive process which did not observably improve accuracy.

4.3.2 Model Architecture

Table 4.1 summarizes the four machine learning classifiers used together with their respective hyperparameters.

Model	Key Hyperparameters
Random Forest	$n\_estimators$=300, criterion=`entropy`, class_weight=`balanced`
Logistic Regression	solver=`liblinear`, C=1.0, max_iter=1 000, class_weight=`balanced`
Multinomial Naive Bayes	α=1.0 (default Laplace smoothing)
Decision Tree	unrestricted depth, criterion=`gini`, class_weight=`balanced`

Table 4.1: Classical ML baselines and their principal settings.

All four models are integrated in a pipeline whose first stage is a TF-IDF vectorizer. The vectorizer keeps the 3.500 most frequent tokens encountered in the processed data and also represents each message using both unigrams and bigrams (meaning it can represent phrases like `"click here"` as a single feature). In addition to that, the vectorizer drops terms that occur in more than 90% or in fewer than five documents, and uses sublinear term-frequency scaling.

The choice of these models was taken after careful consideration; besides their popularity in the ML algorithms landscape, these methods were heavily used in the previously discussed state-of-the-art for text-based phishing classification. LR offers a strong linear baseline for sparse text; RF and DT probe non-linear interactions; NB represents the classic spam-filter heuristic.

4.3.3 Training Strategy

Data split

Ten percent of the training fold is set aside as a validation set during each hyperparameter tuning run for models like LR and RF.

Imbalance handling

With about 60 % legitimate and 40 % phishing emails, the bias is mild, nevertheless, `class_weight=balanced` is enabled for RF, DT and LR to decrease false negatives rate. Experiments with SMOTE oversampling did not show a consistent benefit and were at last removed from the final configuration.

Reproducibility

All random seeds are fixed (random_state = 42), as in the case of the DL training; the full pipeline vectorizer plus trained model is serialized with joblib for determin-

istic reruns. Furthermore, the full pipeline is lightweight: preprocessing (cleaning + vectorization) completes in roughly 3 minutes, and training all four classical models takes about 10 minutes; much more quicker than the deep-learning counterpart.

4.4 BERT pretrained model for URL detection

As stated earlier in the paper, phishing websites often disguise themselves as legitimate domains to deceive users into prompting their credentials or financial data, and thus becoming phishing attack victims. Having that said, URLs are strong indicators of malicious intent and therefore automatically analyzing them using powerful and accurate tools is crucial for preventing cyberattacks.

Regarding the method used for the analysis step, I chose to use a pretrained classification model namely `URLBERT-Tiny v4` [Cra23]. It is a light version of the `URLBERT` architecture, itself being BERT-style pretrained specifically on millions of acrshorturl strings. As mentioned before, it is a lightweight model that occupies a size of only 14.8 MB on disk and uses an Apache-2.0 license. Despite its compact infrastructure, the model is highly optimized for URL classification tasks, and surpasses general-purpose BERT variants because it captures semantics and characteristics that typical language models may overlook.

Briefly described, the model works as follows: it splits on separators such as `/:?=` and in this manner words such as `https`, `paypal`, `com` become tokens, helping the encoder observe suspicious sequences of text. Afterwards, the transformer encoder analyzes the URL as a whole, looking at how different parts relate to each other, performing a so called "self-attention mechanism". Lastly, after "understanding" the full URL, the model takes its summary (the special [CLS] token, which acts as a summary for the entire input) and sends it to a final decision making layer. This layer then outputs a number that tells how likely it is that the URL is malicious.

This model is an essential layer of protection in the phishing detection pipeline by offering a fast and accurate assessment of URLs. Its integration into the system enables real-time URL evaluation, which is further detailed in the implementation chapter.

4.5 Datasets

The trainings of the AI models were conducted on the [Sub24] dataset, available on Kaggle. The dataset contains 18.000 labeled messages, and each entry in the CSV file contains the email's subject and body together with one of the following labels `Phishing Email` or `Safe Email`. The labels are mapped for easier processing to

1 and 0 respectively. The dataset satisfies a commonly cited heuristic in ML field that recommends having several thousand examples per class to enable effective model training. In this case, both phishing and legitimate categories exceed this threshold, allowing the model to learn robust patterns.

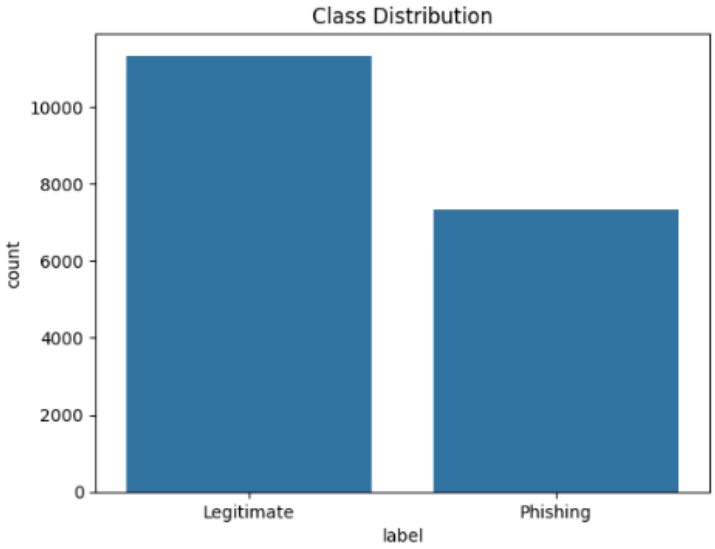


Figure 4.1: Dataset class distribution figure

As observed in Figure 4.1, the data is slightly unbalanced in the favor of legitimate emails over phishing ones. The gap is not large enough to cripple the classifier, while both previously mentioned classes still have thousands of examples, so the model doesn't ignore the minority class during training.

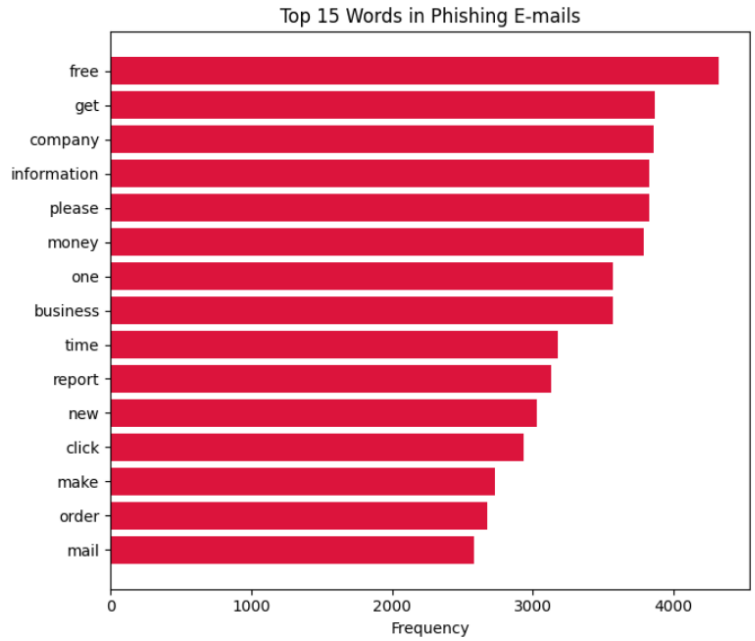


Figure 4.2: Top 15 most frequent words in phishing-labeled emails, extracted after preprocessing.

To better understand the lexical distribution of phishing messages in the dataset, a word-frequency analysis in phishing emails was carried out on the samples. As shown in Figure 4.2, the most frequent terms include strong phishing-indicator words such as 'free', 'money', and 'click', together with more neutral but commonly used terms like 'company' or 'information'. This observation emphasizes the unavoidable mix of obvious phishing cues and context dependent words, which may introduce a certain level of classification bias.

Although the dataset is sufficient for a baseline model, it may still have biases due to the data collection method. For example, phishing emails in this set might follow specific patterns, such as spelling errors or often used indicators of urgency, which may not generalize well to more sophisticated attacks.

As a future improvement, additional datasets from different sources and covering a broader range of phishing strategies and patterns should be incorporated into the training and evaluation pipeline. This would improve the model's stability in more real-world scenarios, including advanced phishing attempts that deviate from the common patterns found in the initial dataset.

Chapter 5

Application Development

5.1 Requirements

To support real-time phishing detection and seamless user integration, the system is built around the following functional and non-functional requirements.

5.1.1 Functional Requirements

- Users will authenticate through **OAuth2**, explicitly granting the necessary permissions for email monitoring.
- The system will monitor a user's Gmail inbox in real time using the **Gmail API** and **Google Cloud Pub/Sub** while also making use of push-based notifications.
- A **browser extension** will be provided for automatic classification, which is performed after about 2 seconds after the user checks an email.
- The system performs **static analysis** on each email, mainly based on the domain utility of the VirusTotal API.
- Emails will be classified using ML or DL models that incorporate header features, content features, and NLP-based content analysis.
- A lightweight **phishing awareness quiz** will be available, presenting users with difficulty categorized questions and storing their answers for future feedback.

5.1.2 Non-Functional Requirements

- The system must ensure low-latency processing of incoming emails and support high availability.

- All components must be modular and extensible to accommodate future integrations or new detection methods.
- Sensitive user data must be handled securely, user should be informed about the application's desired permissions and consent their usage.
- The browser extension must be lightweight, intuitive, and non-disturbing to user experience.
- All relevant metadata, analysis results, and API calls will be properly logged in the backend.
- The system must respect secure coding practices and protect against basic vulnerabilities(Cross-Site Scripting, SQL injection)

5.2 System Design

The application was designed with three key software engineering principles in mind: maintainability, scalability and extensibility, ensuring that it remains adaptable, robust, and easy to evolve over time.

Maintainability

Maintainability is guaranteed by the presence of a modular structure, separating core functionalities into components that communicate between them. Furthermore, the app employs a layered-architecture, more specifically a three-tier-architecture:

- **Presentation layer**
- **Application layer**
- **Data layer**

In addition, a logging module to ease debugging and information gathering, backed up by a configuration manager for centralized management of environment variables and service settings, provide a strong foundation for a maintainable and durable application.

Scalability

Scalability is ensured by the event-driven architecture and the extensive usage of external services such as Gmail API, Google Cloud Pub/Sub or Web Push Notifications. Scalability comes naturally because the system works like a relay team:

when a new e-mail arrives, a tiny service hands the job off to Google's Gmail API, puts a message on Google Pub/Sub, and then workers pick it up if they're needed. Analysis results and the actual e-mail get stored in the database for effortless later usage, and also travel back to users through web-push alerts, ensuring to alert the user even if it is not actively using the web application. Each piece can grow or shrink on its own, so the entire app remains smooth under any circumstances.

Extensibility

Extensibility is achieved by Object Relational Mapper(ORM) usage, standardized RESTful endpoints and easily pluggable AI models.

- An ORM maps database relations(tables) to Flask classes, making the creation and extension of entities more intuitive and easily done.
- Standardized endpoints have the following features: consistent HTTP methods(GET, POST, PUT, DELETE), clean input and output data(JSON format) and well-defined return codes(200, 401, 422).
- The models used in the application are abstracted behind the service layer, making their replacement or upgrade seamless.

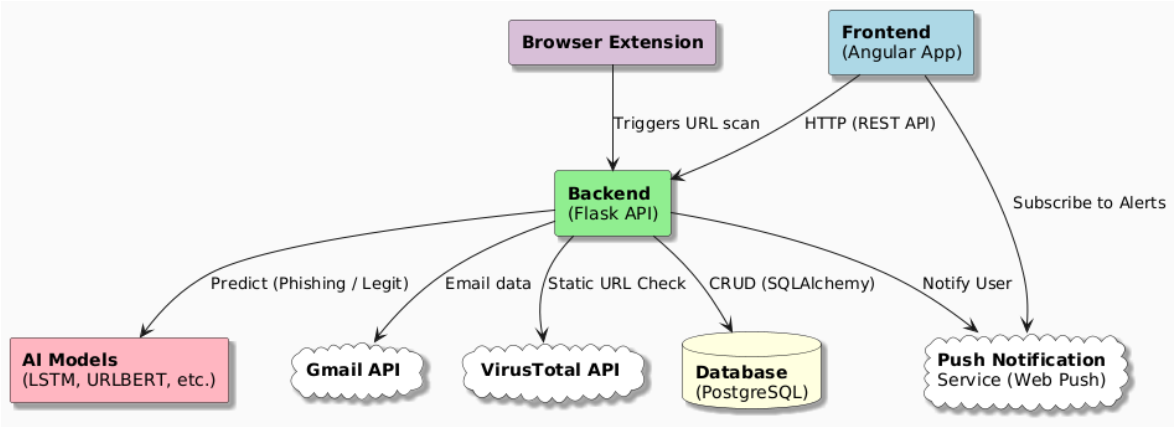


Figure 5.1: System Architecture Diagram

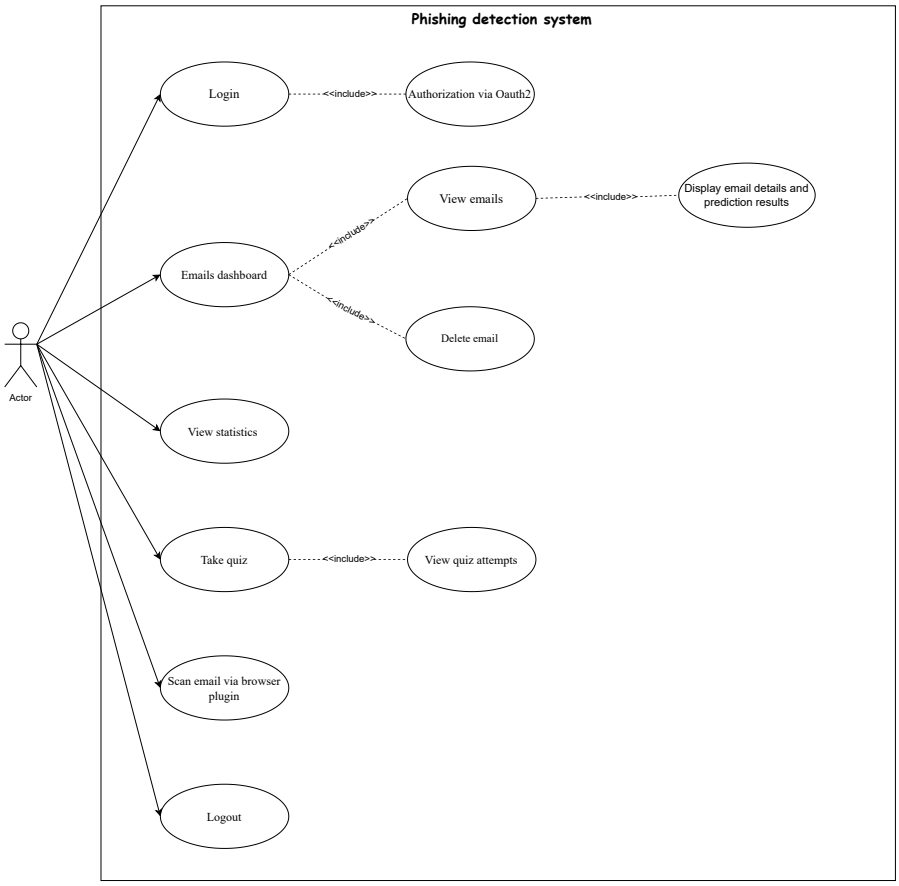


Figure 5.2: Use Case Diagram

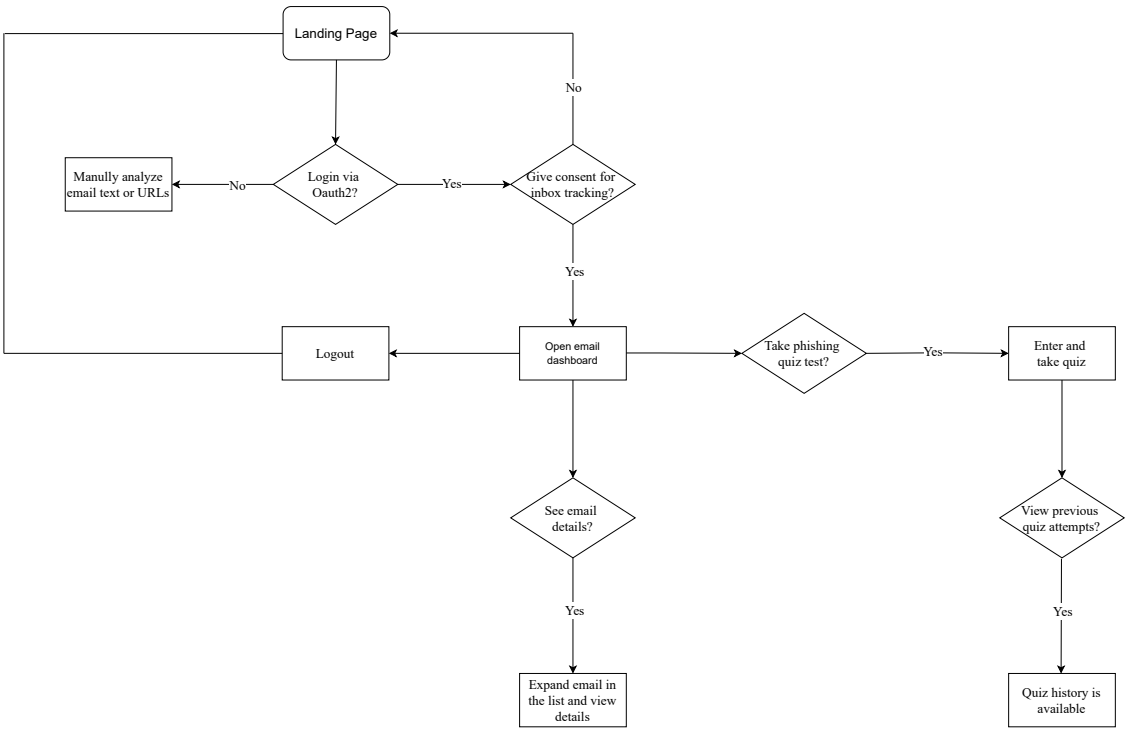


Figure 5.3: Application Flow Diagram

5.3 Implementation

The back-end application is built in **Flask**, a lightweight Web Server Gateway Interface(WSGI) based on the Python programming language. It offers a flexible, extension-rich environment and a REST-friendly infrastructure, making it the ideal choice for building fast, clean, and responsive RESTful APIs.

Some worth mentioning libraries used in the application development:

- **SQLAlchemy** Provides a flexible ORM which enables easy data storage and facilitates querying scanned emails, predictions, and user data using Python models.
- **Google Auth OAuthlib** Enables secure Gmail integration by handling OAuth 2.0 authentication flows for accessing user inbox data.
- **PyWebPush** Allows the app to send real-time browser notifications for newly detected phishing emails using the Web Push protocol.
- **Transformers** Complements the phishing detection logic by providing pre-trained NLP models like BERT for analyzing URLs.
- **Torch** Supports deep learning tasks within Transformers-based models, enabling efficient inference and fine-tuning if needed.
- **Google Api Python Client** Empowers connection to Gmail's API to fetch emails, receive push notifications, and analyze inbox contents.

5.3.1 Database

In the context of software projects, databases represent an organized collection of data managed by a database management system (DBMS). They are a crucial utility that ensure data persistency in a software solution, meaning that once the data is inserted and present in a database, it will remain there under any circumstances(system crashes, system reboots, user log offs). Besides the crucial feature of data persistency, modern DBMS offer robust data definition and manipulation, security and access control and ACID(**A**tomicity, **C**onsistency, **I**ntegrity, **D**urability) compliant transactions.

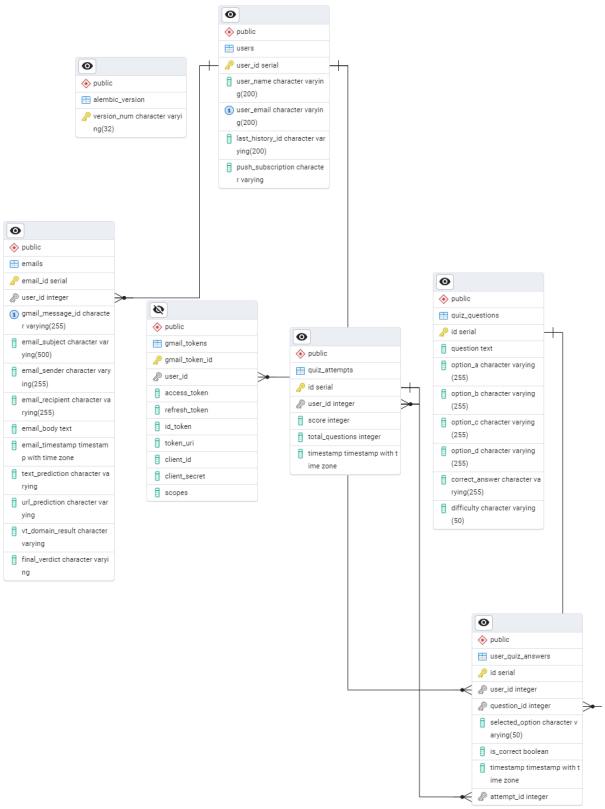


Figure 5.4: Entity-Relationship diagram of the application's database

As depicted in Figure 5.4, the PostgreSQL database contains data regarding users, emails, quiz questions and other related data. The tables can be described as follows:

- **emails**: Contains email information such as subject, sender, recipient, body, timestamp, classifications from AI model and VirusTotal API, and the final verdict. Key columns include email_subject, email_sender, email_body, text_prediction, and final_verdict.
- **users**: Stores user identity and metadata including name, email, Gmail refresh token, and push notification subscription. Key columns are user_email, user_name, refresh_token, and push_subscription.
- **gmail_tokens**: Stores OAuth2 credentials used to access Gmail inboxes. Includes tokens, client credentials, and scope. Columns include access_token, refresh_token, token_uri, and scopes.
- **quiz_questions**: Represents the phishing quiz question table. Each record contains the question text, four options, the correct answer, and difficulty level. Includes columns like question, option_a, option_b, correct_answer, and difficulty.

- **quiz_attempts**: Tracks quiz sessions per user, storing score, total questions, and timestamp. Key fields are user_id, score, total_questions, and timestamp.
- **user_quiz_answers**: Stores each individual answer selected during a quiz attempt. Contains the selected option, whether it was correct, and links to the question and attempt. Important fields are selected_option, is_correct, question_id, and attempt_id.

5.3.2 Security

JSON Web Tokens(JWT) are used as a secure method for authenticating and authorizing users after they have logged in via Gmail OAuth2. Once the user completes the OAuth2 flow and their identity is verified through Google's API, a signed JWT is provided to the client. The token encapsulates user data and is the bridge between allowing the frontend to make authenticated requests to the protected backend routes without needing re-authentication each time. In this system, both an `access_token` and a `refresh_token` are used: the former grants short term access, while the latter allows for issuing new access tokens without requiring the user to log in again. JWTs ensure stateless and scalable session management which also enforcing access control based on the validity of the token.

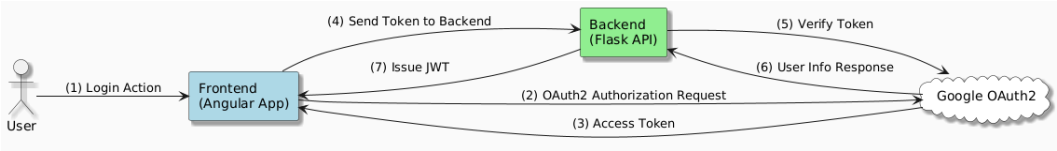


Figure 5.5: JWT authentication and authorization flow

Figure 5.5 illustrates the authentication flow using OAuth2 and JWT, showing how tokens are used and verified for secure communication between the frontend and backend in the developed application.

5.3.3 Email monitoring system

Gmail API

Emails are digital objects which contain data that is transmitted over the internet, enabling users to exchange text, attachments, and multimedia files. Email providers are services that have the responsibility of facilitating the email handling process: Gmail, Microsoft Outlook, Apple Mail. Of these, Gmail is by far the most popular service, managing to peak at astonishing usage numbers: 121 billion emails exchanged every day and 2.5 billion active Gmail users as per [EE25], motivating the

choice of using Gmail as a mail service for the developed application. Gmail API was used to achieve 2 key aspects of the application: it facilitates secure Oauth2-based user authentication for safely accessing individual inbox data and secondly, it provides real-time inbox monitoring through a Publisher/Subscriber notification mechanism, allowing the architecture to respond very fast to new emails received.

```
auth_flow = Flow.from_client_secrets_file(
    GmailConfig.get_secret_file_path(),
    GmailConfig.GMAIL_SCOPE,
    redirect_uri=GmailConfig.REDIRECT_URI
)

auth_url, state = auth_flow.authorization_url(
    access_type="offline",
    prompt="consent",
    include_granted_scopes="true"
)
```

Listing 5.1: Gmail Oauth Flask implementation

The code snippet Listing 5.1 initializes the OAuth2 authorization flow using the Gmail API. It generates a secure authorization URL and manages user consent, allowing the application to safely request access to inbox data.

```
service = build('gmail', 'v1', credentials=creds)
profile = service.users().getProfile(userId='me').execute()
email = profile['emailAddress']
user = gmail_service.get_user_by_email(email)

# Service for watching the inbox gmail of the user using Gmail API
watch_request_body = {
    "labelIds": ["INBOX"],
    "topicName": GmailConfig.GMAIL_SUBSCRIPTION_TOPIC
}
watch_response = service.users().watch(userId='me', body=
    watch_request_body).execute()
```

Listing 5.2: Inbox Watcher Flask implementation

Listing 5.2 sets up a watcher on the user's inbox via Gmail API, relying on Google Cloud Pub/Sub service under the hood. It subscribes to Gmail push notifications for new messages under the `INBOX` label, so that the app is notified whenever the Gmail inbox is updated, and more specifically when the user receives a new email. This feature is how real-time email monitoring is achieved in the full stack application.

Integration with AI models

Each incoming email is automatically processed through multiple layers of detection mechanisms. The system uses three complementary techniques: a deep learning model for predicting phishing based on the email's textual content, a pretrained URL classification model to assess embedded links, and a static threat intelligence from the VirusTotal API, which evaluates domain reputation.

To minimize false negatives, a conservative decision rule was applied: *if any of the three detectors flags the email as suspicious or phishing*, a browser-level alert is immediately triggered and the email final verdict is marked as "phishing". This multi-layered approach balances precision and coverage, helping to catch a wide range of phishing techniques while maintaining performance in real time.

Figure 5.6 illustrates the specific pipeline for URL classification using the URLBERT-Tiny model, one of the core components in the overall detection strategy. The flow includes URL preprocessing, vectorization using transformer encoder, and prediction, with results stored and then displayed in the frontend.

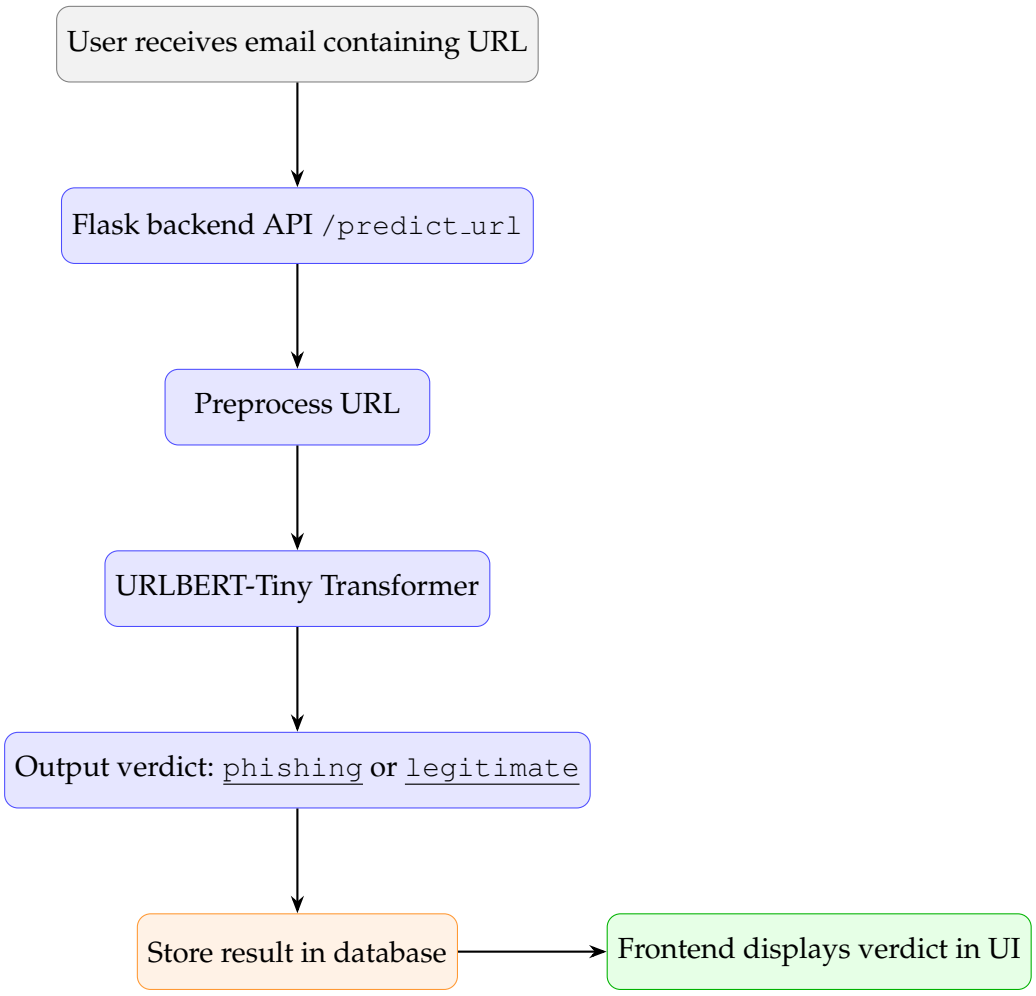


Figure 5.6: Phishing URL detection pipeline

Push-web notifications

Push notifications were implemented as part of my contribution to building a complete and automated system capable of real-time user interaction. This feature allows the backend to send targeted, browser-level messages to authenticated users, even when the application is not actively open on their device. By using the `Push API`, which is part of the Web Push Protocol, enabling a service worker running in the background ready to receive messages from a remote server. By securely managing subscriptions and authenticating messages with Voluntary Application Server Identification(`VAPID`) keys, the system achieves efficient and reliable communication without requiring active and continuous fetching. This integration boosts the responsiveness and completeness of the overall system architecture, aligning with the project's goal of delivering a robust and scalable platform.

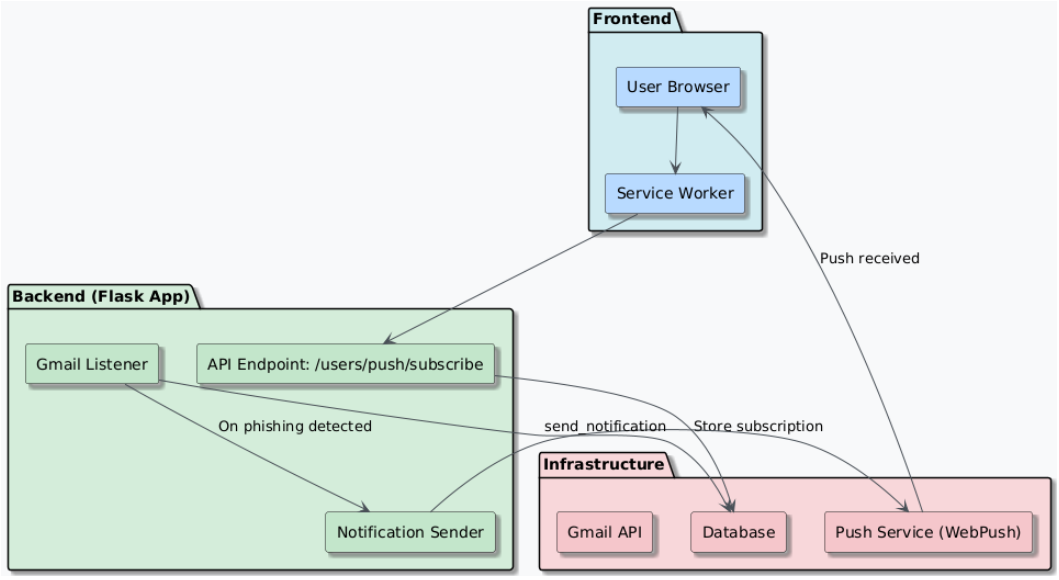


Figure 5.7: Flow of the push notification system in the phishing detection architecture

Figure 5.7 shows the push notification flow in the fullstack application. When a phishing email is detected, the backend sends a message through the Web Push Service using stored browser subscriptions. These subscriptions are registered via an endpoint `POST` request and saved in the database. A service worker in the user's browser receives the push, enabling real-time alerts even when the app is inactive.

The design and implementation of the real-time monitoring system including Gmail integration, AI-based classification, and browser push notifications represent my original contribution to the development of a responsive and automated phishing detection platform.

5.3.4 Quiz section implementation

The quiz part of the application has the purpose of raising user-awareness in the field of phishing. A collection of questions and real-world examples has been gathered from security blogs and corporate training case studies, such as Cisco's phishing education page [Cis] and KnowBe4's security awareness tools [Kno], with the goal of composing a balanced mix of easy, medium and hard scenarios that reflect as well as inbox threats and also important theoretical concepts. Furthermore, building some of the questions was the result of original ideas and phishing scenarios which I composed.

Code-wise, it has a straight-forward implementation which is easy to use. The user can click on the `Take quiz` button and he will be redirected to a page where 10 questions are displayed, which are shuffled on every run. These are fetched from the database using the backend API `/quiz/questions`. The user then completes the quiz, and when clicking the "Submit" button the answers are sent to the backend through a POST request to the endpoint `/quiz/submit`. The backend checks the answers and sends back the result obtained by the user. In addition, learners can also open `Quiz History` page at any time, powered by the HTTP call GET and using the API `/quiz/history` to see all their past scores and keep track of how much they have improved.

5.4 Web Application

In this chapter, the frontend features of the application will be described, which can be split into two principal branches: the **web interface** built in Angular and the **browser plugin**.

5.4.1 Angular-based web interface

The front end is a single–page application written in Angular 19. Once the user completes Google OAuth2 authentication, the browser loads a bundle page that communicates with the Flask API through HTTP requests. Page transitions are therefore very fast, since Angular swaps view components in memory instead of performing full reloads.

Core components

The solution proposes a web interface which is structured into modular components, each serving a specific role in the application:

- **Dashboard** The dashboard component displays the received emails since first logging into the application, thus representing a master-detail of the scanned emails. The master part shows a paginated list of email summaries containing subject, sender and recipient. Further, the detail part occurs when one of the email entities is clicked, revealing more information such as receival timestamp, body content and prediction results, and also a *Report as False Positive* button.
- **Manual Analyze** (Landing page) This section is the entry point of the application, where the user can choose to proceed with OAuth2 authentication and also manually input and submit a custom email text and URL for phishing analysis.
- **Quiz** Provides interactive, single-choice quizzes to educate users on phishing attacks. These quizzes are based on real phishing examples and general scenarios. Each question has a label which describes its difficulty (easy, medium or hard).
- **Quiz History** Shows a historical view of all quizzes completed by the user, including scores and time of completion. The purpose of this component is for the user to be able to keep track of its progress.
- **Statistics** Displays visual analytics of system usage and detection performance over time. This includes pie charts for prediction ratios and a timeline of received emails with their corresponding classification.

Shared components

To ensure consistency and reusability, shared components are used across the web application. `Navbar` provides intuitive navigation between application components, while also including the possibility of logout, backed up by a confirmation dialog. Furthermore, the `EmailCard` component focuses on an uniformed manner through which individual emails are used across views. Additionally, models such as `Email`, `Prediction`, and `QuizAttempt` define strongly-typed interfaces used throughout the app, ensuring consistent data structures when interacting with backend services.

Services

The application relies on a service-oriented architecture to achieve separation of concerns and API communication.

Auth service has the role of managing the authentication state of the user. It handles JWT token storage and user session expiration.

Gmail service encapsulates interactions related to emails with the backend, including the manual analysis of text or URL and fetching the updated list of inbox emails.

User Interface/User Experience features

The application emphasizes user-friendliness, clarity, and responsiveness, which are achieved by respecting some basic fundamentals of web development. Those include responsive design, component reusability, and separation of concerns, all integrated in the frontend implementation. Moreover, pagination improves performance and usability for large datasets of emails and visual feedback is prioritized through the use of loading indicators, snackbars and suggestive color cues, boosting interactivity and clarity on the user side.

5.4.2 Browser Extension-Based Detection

In addition to the fullstack web interface, a lightweight browser extension was developed for real-time phishing detection inside the Gmail web interface. Developed in the JavaScript programming language, this approach focuses on improving usability and ease of access to the solutions developed in the backend, without requiring the users to navigate away from their inbox.

The extension injects a script directly in the Gmail pages, where it is actively monitoring email openings. When an email is opened by the user, the JavaScript code asynchronously extracts the subject, sender, and body from the Document Object Model (DOM) as it can be observed in the code snipped from 5.3.

```
// Select the DOM elements that contain relevant email fields
const subjectNode = document.querySelector('h2[data-legacy-thread-id]');
const bodyContainer = document.querySelector('div.a3s');
const senderNode = document.querySelector('.gD');

// Extract the actual content from DOM nodes
const subject = subjectNode?.innerText || '';
const sender = senderNode?.getAttribute('email') || senderNode?.innerText
    || '';
let body = bodyContainer?.innerHTML || '';
```

Listing 5.3: Extracting email fields from Gmail DOM

In addition, the data is sent to the backend via a `POST` request using the `fetch()` API. The operation is handled using Promises, ensuring the web interface remains

responsive while waiting for the server's phishing prediction. The backend then responds with a structured prediction result, including a final verdict and the result of the 3 layers of classification it goes through: text-based, URL-based and a domain scan using VirusTotal API.

At last step, when the verdict is received by the browser plugin, it is displayed above the email subject, in a visually appealing manner. It indicates if the message was classified as legitimate, phishing or unknown using suggestive colors for each of the responses and an option to report the email as a false positive reflected with a button. This feature can be visually observed in 5.8 below.

(a) Phishing detection result: **Potential Phishing**

(b) Phishing detection result: **Likely Safe**

Figure 5.8: Comparison between phishing and safe email detection results shown in the browser plugin.

Compared to the Angular frontend, the extension offers a minimal, zero-setup experience, better suited for casual or non-technical users. While it does not provide historical email tracking or analytics, it excels in real-time usability, especially for quickly assessing individual emails.

The extension is publicly available on the **Chrome Web Store**, making it accessible to any user who wishes to enhance their email security with no setup necessary, only permission to access the content of an email when it is opened.

This browser extension approach proves that phishing detection can be integrated easily into a user's existing workflow, enabling faster reaction times and better accessibility. The design and development of this browser extension represent an original contribution of this thesis, reflecting practical innovation in real-time phishing detection.

5.5 Testing

Testing is a key aspect of software development that ensures the correctness, consistency, and reliability of the application. In the context of the phishing detection application development, testing played a crucial role in validating and ensuring a correct flow for various components such as phishing classification, Gmail integration, and user interactions through APIs.

Testing approach The Flask application is tested using the libraries `pytest` for simplifying writing unit and integration tests and `flask.testing.FlaskClient`

for simulating HTTP requests in tests.

Unit testing was used for testing isolated pieces of code, most commonly service functionalities. A relevant example for the project is the email text classification test shown in Listing 5.4.
This test focuses on the functionality of the service responsible for classifying input text. By mocking the actual logic behind `predict_email_text_direct`, the test ensures that the `/emails/predict_text` endpoint correctly handles inputs and returns a well-structured prediction.

```
def test_predict_text(client, mocker):
    mock_prediction = {'label': 'phishing', 'confidence': 0.92}
    mocker.patch(
        'routes.email_routes.email_service.predict_email_text_direct',
        return_value=mock_prediction
    )

    response = client.post('/emails/predict_text', json={
        'text': 'You have won a 50% discount on our product.
            Click here urgently to claim it!'
    })

    assert response.status_code == 200
    data = response.get_json()
    assert 'prediction' in data
    assert data['prediction']['label'] == 'phishing'
```

Listing 5.4: Email text classification unit test

Integration testing validates interaction between various modules of the application and how they operate together. Listing 5.5 demonstrates this through the testing of email URL classification. This test simulates a real interaction by including an authenticated header and sending a POST request with a JSON payload. The service method `predict_url` is mocked, but the test still validates the complete API endpoint behavior.

```python
def test_analyze_email_url(client, auth_header, mocker):
    mock_data = {
        "url": "http://phish.test"
    }

    mocker.patch(
        'routes.email_routes.email_service.predict_url',
        return_value={"prediction": "phishing"}
    )

    response = client.post('/emails/predict_url',
                           data=json.dumps(mock_data),
                           headers={**auth_header, "Content-Type": "
                               application/json"})

    assert response.status_code == 200
    data = response.get_json()
    assert data['prediction'] == 'phishing'
```

Listing 5.5: Email URL classification integration test

Manual testing was performed mainly for frontend operations and simple workflows, most notably: User Interface interactions with various components(buttons, forms), alert management, and browser push notifications. Furthermore, the application's RESTful API endpoints were manually tested using **Postman**, a tool that allows developers to construct, send, and inspect HTTP requests. This facilitated the verification of backend responses, JWT authentication, and error handling before the frontend integration.

5.6 Deployment

To make the application publicly accessible, it was deployed to a cloud platform using containerization. The container is built and pushed to *Railway*, a managed Platform as a Service (PaaS), chosen for its easy Continuous Integration/Continuous Deployment pipelines, built-in PostgreSQL add-on, and sealed "Variables" store for managing secrets.

The entire application is shipped as a **single Flask monolith**. Every feature of the application: Gmail feature, phishing analysis, quiz section and REST API are inside one Docker container and share a single PostgreSQL instance. This approach avoids the network overhead and operational complexity of a micro-services architecture, which was not necessary in the current application context.

5.6.1 Database deployment

All persistent data (users, emails, quiz attempts) reside in PostgreSQL, provisioned as a managed database on Railway. Connections are made through `SQLAlchemy` with parametrized queries, eliminating the risk of SQL injection vulnerability. Railway takes daily encrypted snapshots, and furthermore, row-level security and least privilege roles restrict access to the service account only. Because the the database runs in the same private Virtual Private Cloud(VPC) as the container, data never traverses the public internet unencrypted.

5.6.2 Backend deployment

The backend is containerized with a lightweight `Dockerfile` built on the python:3.10-slim base image: it installs project dependencies and fetches required language assets for preprocessing, then starts `gunicorn` with 4 workers. The image is published to Railway's database and can be manually redeployed on every push to the main branch. Environment variables (`OAuth secrets`, `DB URL`, `JWT key`) are stored in Railway's encrypted *Variables* project dashboard, never hard-coded or pushed to Git so that each deployment pulls its secrets securely at runtime.

5.6.3 Frontend deployment

The frontend of the application, built in Angular, is deployed as a separate service in the Railway infrastructure alongside the backend and the database services. After running the Angular build command `ng build`, the resulting `dist` folder contains the production ready files. The frontend deployment process ensures that the built application is accessible over a public network, permitting the users accessible User Interface interactions.

5.7 Application flows

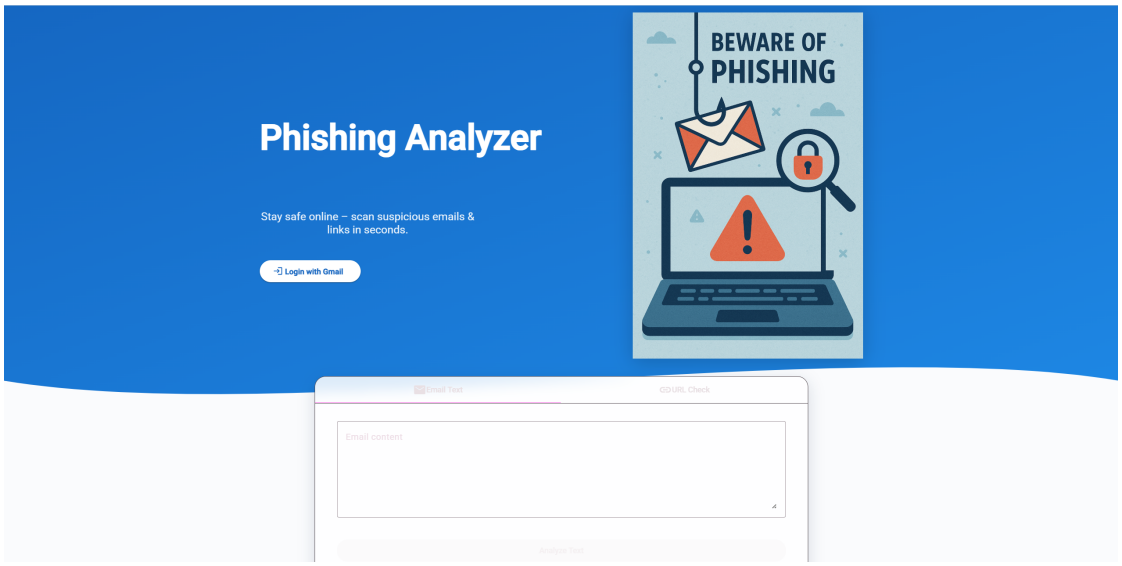


Figure 5.9: Web application landing page

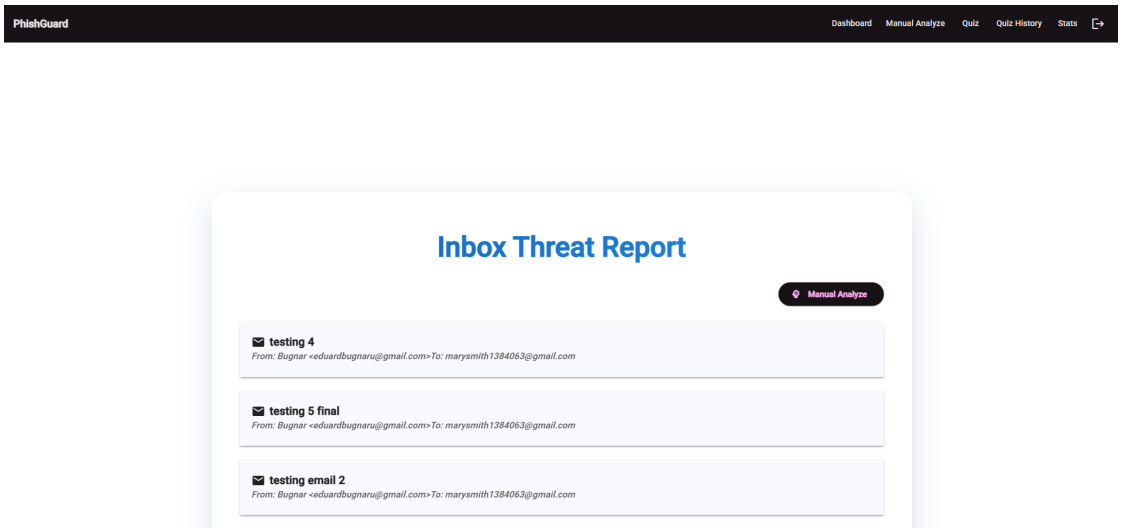


Figure 5.10: Inbox Dashboard page

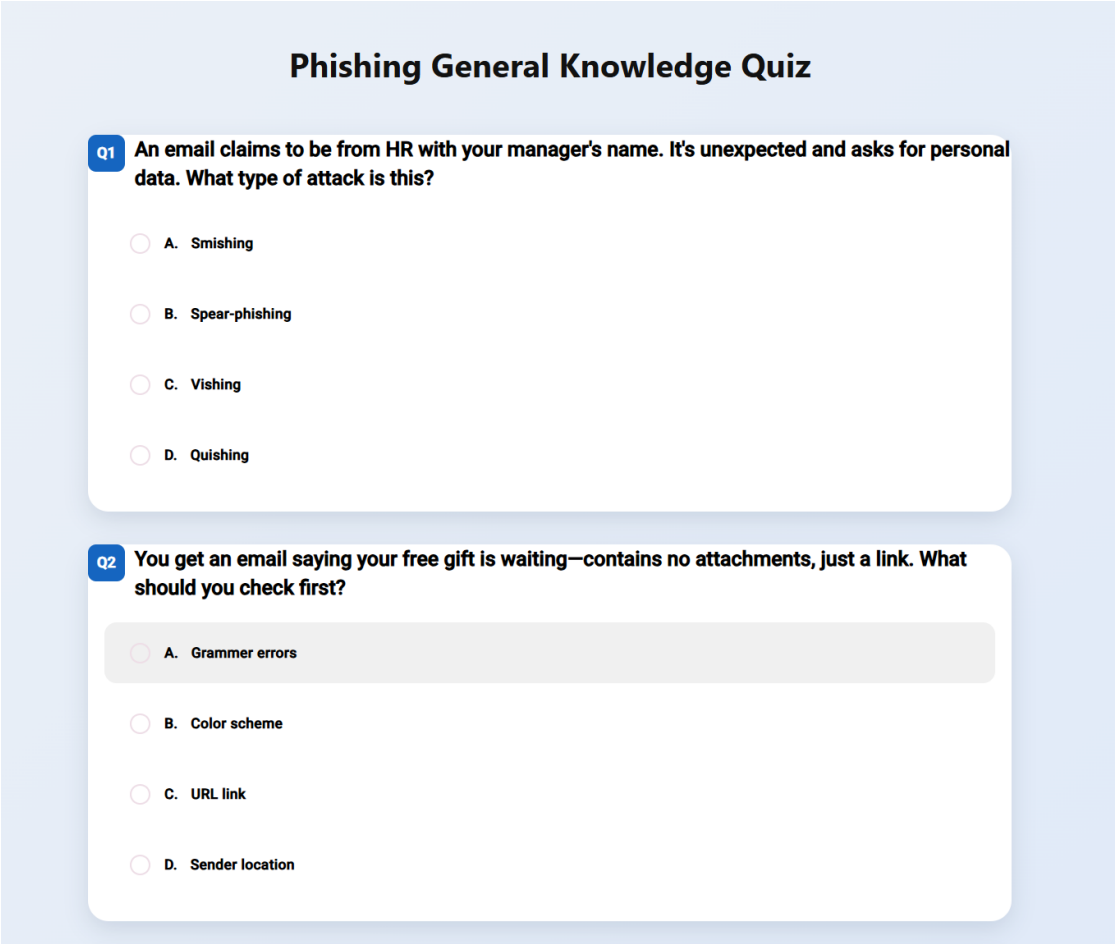


Figure 5.11: Take-quiz page

Figure 5.12: Quiz history page

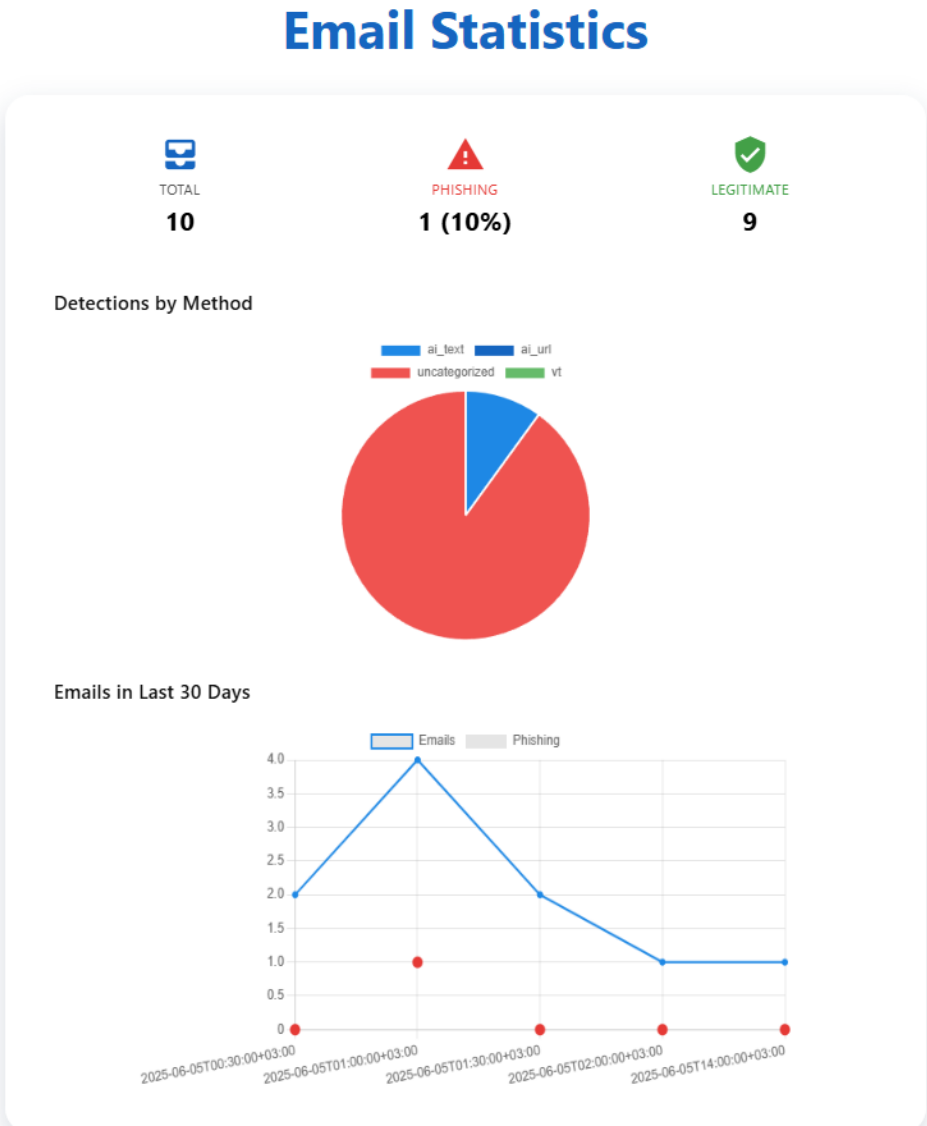


Figure 5.13: Statistics page

Chapter 6

Results analysis

6.1 Artificial Intelligence models classification metrics

6.1.1 Overview

Detecting phishing emails is a binary text-classification problem in which the positive class (phishing) is noticeable rarer than the negative one (legitimate).

To ensure fair assessment, the dataset corpus was divided 80/10/10 into training, validation and test partitions using sampling, thus preserving the class ratio in all subsets. All quantitative results reported below come from the unseen *test* split unless explicitly noted.

- **Accuracy** $= \frac{TP+TN}{TP+TN+FP+FN}$ - represents the overall correctness rate of the model.
- **Precision** $= \frac{TP}{TP+FP}$ - proportion of predicted-positive emails which are truly phishing.
- **Recall** $= \frac{TP}{TP+FN}$ - proportion of phishing emails that are successfully detected by the model
- **F-Score** $= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ - harmonic mean of the two evaluation metrics above(Recall and F-Score).

These scalars are easy to interpret and allow side-by-side comparison of the traditional machine learning baselines (RF, LR, SVM, DT) with the deep LSTM model.

6.1.2 Metrics aggregation

Table 6.1 reflects the comparison of the ML algorithms mentioned previously and the DL model used.

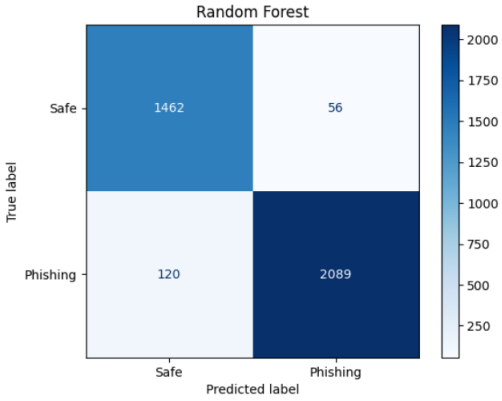
Model	Accuracy	Precision	Recall	F1
Random Forest	0.9528	0.9241	0.9631	0.9432
Logistic Regression	0.9616	0.9338	0.9750	0.9539
Naive Bayes	0.9431	0.9240	0.9374	0.9307
Decision Tree	0.9047	0.8678	0.9038	0.8854
LSTM	**0.9667**	**0.9454**	**0.9713**	**0.9582**

Table 6.1: Performance comparison of classical ML models and the LSTM deep-learning model

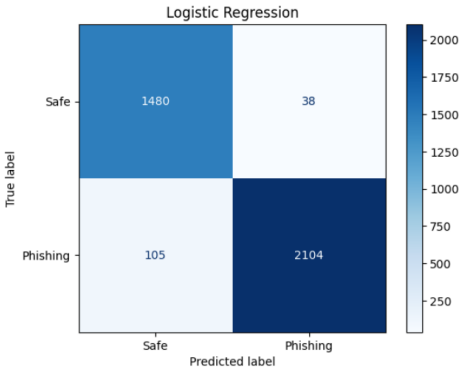
It is clearly observed that the LSTM outperforms all classical models, achieving the best balance of precision (95 %) and recall (97 %), so it catches most phishing emails with few false alarms. Logistic Regression is visually described as the strongest Machine Learning algorithm: only around 0.5 points lower than the LSTM in F1 proving it can be considered a lightweight alternative. Random Forest and Naive Bayes still deliver a 94% accuracy, whereas a single Decision Tree falls behind all other algorithms noticeably.

6.1.3 Confusion Matrix Assessment

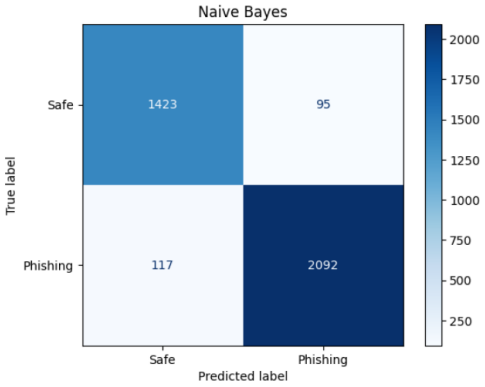
A confusion matrix breaks the test predictions into *true positives* (bottom-right), *true negatives* (top-left), and the two error types: *false positives* (top-right) and *false negatives* (bottom-left). For a phishing detection architecture context, false negatives are the most costly (missed attacks), since they can directly impact the user, and a moderate number of false positives is acceptable. The following comparison in Figure 6.1 highlights how each model handles this trade-off between detection sensitivity and false alarm rate.



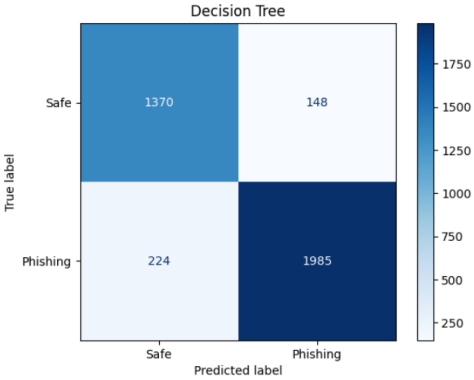
(a) Random Forest



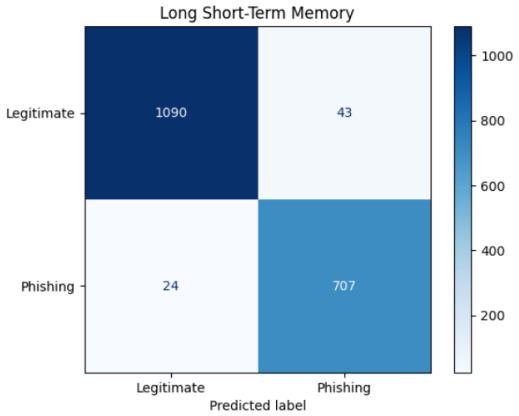
(b) Logistic Regression



(c) Naive Bayes



(d) Decision Tree



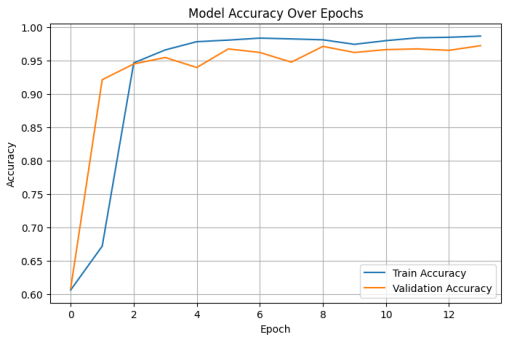
(e) Long Short-Term Memory

Figure 6.1: Confusion matrices for the four classical machine learning models (top two rows) and the LSTM deep learning model (last row).

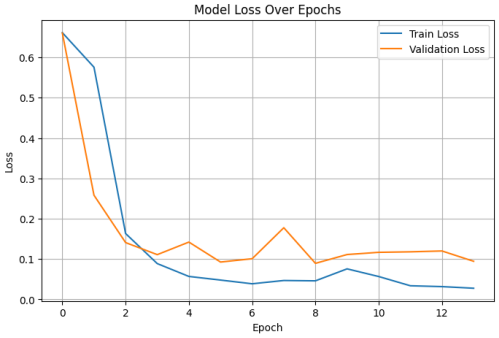
Overall, the confusion matrix diagrams from above reinforce the earlier model ranking: the LSTM maximizes attack catch rate while keeping the false positive frequency the lowest among all models.

6.1.4 Deep-Learning Training Curves

Classical ML models are trained in a single pass (no epoch loop), so loss curves are not applicable. Instead, their performance is summarized by the scalar test metrics and confusion matrices presented earlier, which allow a fair comparison with the LSTM's final epoch (more precisely the epoch early stopping chooses).



(a) Model accuracy over epochs (train vs. validation)



(b) Model loss over epochs (train vs. validation)

Figure 6.2: Learning curves for the LSTM classifier.

The training curves in Figure 6.2 show that the LSTM model learns rapidly, reaching over 95% accuracy within the first few epochs. Both training and validation accuracy remain high and closely placed, suggesting good generalization and low overfitting. Similarly, the loss curves indicate a sharp initial decrease and stable convergence. Minor fluctuations in validation loss are present but do not indicate instability. Therefore, the model demonstrates a strong and reliable performance across training iterations.

Given the previous analysis results and remarks, it can be concluded that the LSTM model consistently outperforms traditional ML models across all evaluation metrics, especially in balancing high recall with precision. Given the real-time requirements of phishing detection systems, its ability to generalize well on unseen data while minimizing false negatives makes it the best choice for integration in the system architecture. These comparisons and resulting conclusions represent an original contribution to the system development, supporting the selection of the most effective model for deployment.

Chapter 7

Conclusions

The thesis aimed to demonstrate that an integrated, human-centered approach can raise the bar for phishing defence without relying on massive enterprise-scale resources. At its core are lightweight, text-based detection models, transformer variants fine-tuned for phishing scenarios which proved both reliable and adaptable across various situations. Complementing this layer are two additional safeguards: a pretrained URL classification model to flag malicious links, and integration with the VirusTotal API to enrich domain analysis with external threat intelligence. Together, these layers form a balanced and effective multi-layered defense.

The system provides real-time protection by automatically scanning email and chat traffic and passing them to the three protection layers mentioned above, while also showing alerts in a dashboard. Moreover, it includes a quiz module that turns real phishing examples into quick challenges to train users and raise their awareness in the field.

On the client side, the browser plugin embeds the same language models locally, while providing intuitive risk messages before any dangerous action is taken by the user. Together with the backend monitor and the quiz section of the application, the plugin contributes to a cohesive system: *detect*, *inform*, and *reinforce*; where detection and alerts occur via the plugin, and reinforcement happens through the separate quiz module in the web application.

While results show that a compact, multi-layered defense is practical, there are still limitations. The training data is smaller than state-of-the-art experiments, so some attack scenarios may be missed. The models prioritize speed and efficiency over top-tier accuracy, and real-world testing was limited, thus longer deployments are needed to prove reliability and user trust.

Even with these limitations, the work demonstrates that carefully engineered, intercommunicating components, monitoring, and user-education can produce a system which makes phishing both harder to launch and easier to spot.

7.0.1 Future improvements

Future iterations of the implementation should focus on expanding language coverage by including more diverse and multilingual phishing datasets, enabling the models to generalize better across different linguistic. Improving the AI models themselves, either through more sophisticated architectures or ensemble techniques could enhance accuracy without compromising speed. The dataset used for training and evaluation should be continuously updated and enlarged, incorporating latest phishing strategies.

Additionally, user feedback should be actively integrated into the system, particularly reports of false positives or missed detections. Although the current feedback loop exists, its practical use is irrelevant; future work should emphasize refining this mechanism so that user submitted corrections directly inform model retraining or rule modifications. This improvement would create a more adaptive system that learns not only from chosen data but also from real-world interactions.

Bibliography

[AA23] Samer Atawneh and Hamzah Aljehani. Phishing email detection model using deep learning. *Electronics*, 12(20), 2023.

[AAAA25] Abeer Alhuzali, Ahad Alloqmani, Manar Aljabri, and Fatemah Alharbi. In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets. *Applied Sciences*, 15(6), 2025.

[AAAM+23] Zainab Alshingiti, Rabeah Alaqel, Jalal Al-Muhtadi, Qazi Emad Ul Haq, Kashif Saleem, and Muhammad Hamza Faheem. A deep learning-based phishing detection system using cnn, lstm, and lstm-cnn. *Electronics*, 12(1), 2023.

[AATAA24] Najwa Altwaijry, Isra Al-Turaiki, Reem Alotaibi, and Fatimah Alakeel. Advancing phishing email detection: A comparative study of deep learning models. *Sensors*, 24(7), 2024.

[and24] Er. Kritika and. A comprehensive literature review on phishing url detection using deep learning techniques. *Journal of Cyber Security Technology*, 0(0):1–29, 2024.

[CAA+23] Cybersecurity, Infrastructure Security Agency, National Security Agency, Federal Bureau of Investigation, Multi-State Information Sharing, and Analysis Center. Phishing guidance: Stopping the attack cycle at phase one. Technical report, Joint CISA / NSA / FBI / MS-ISAC, October 2023. Version published October 2023.

[Cis] Cisco. What is phishing? Available at: https://www.cisco.com/site/us/en/learn/topics/security/what-is-phishing.html.

[Cra23] CrabInHoney. urlbert-tiny-v4-phishing-classifier. https://huggingface.co/CrabInHoney/urlbert-tiny-v4-phishing-classifier, 2023.

[CSH+24] Pundalik Chavan, Hassan Yakub Sait, P. Hemanth, Jonathan S. Paul, and Justin A. Deodhar. Comparative analysis of machine learning algorithms for text-based phishing detection. *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, April 2024. Paper Id: IJRASET60366; Publish Date: 2024-04-15.

[DGV24] Giuseppe Desolda, Francesco Greco, and Luca Viganò. APOLLO: A gpt-based tool to detect phishing emails and generate explanations that warn users. *arXiv*, 2024.

[EE25] Cai Ellis and Charlotte Evans. How many emails are sent per day? (fresh data for 2025). EmailTooltester.com, April 2025. Accessed: 2025-05-14.

[HFA24] Qazi Emad ul Haq, Muhammad Hamza Faheem, and Iftikhar Ahmad. Detecting phishing urls based on a deep learning approach to prevent cyber-attacks. *Applied Sciences*, 14(22), 2024.

[JGST20] Daniel Jampen, Gürkan Gür, Thomas Sutter, and Bernhard Tellenbach. Don't click: towards an effective anti-phishing training. a comparative literature review. *Human-centric Computing and Information Sciences*, 10:14–18, 12 2020.

[KCD20] Abhishek Kumar, Jyotir Moy Chatterjee, and Vicente García Díaz. A novel hybrid approach of svm combined with nlp and probabilistic neural network for email phishing. *International Journal of Electrical and Computer Engineering (IJECE)*, 10(1):486–493, Feb 2020.

[Kno] KnowBe4. Free phishing security test. https://www.knowbe4.com/free-cybersecurity-tools/phishing-security-test.

[NRSS25] Alex Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile. NIST Special Publication 800-61r3, National Institute of Standards and Technology, April 2025.

[SBDD19] Ozgur Koray Sahingoz, Ebubekir Buber, Onder Demir, and Banu Diri. Machine learning based phishing detection from urls. *Expert Systems with Applications*, 117:345–357, 2019.

[SGVS21] Said Salloum, Tarek Gaber, Sunil Vadera, and Khaled Shaalan. Phishing email detection using natural language processing techniques: A literature survey. *Procedia Computer Science*, 189:19–28, 01 2021.

[SGVS22] Said Salloum, Tarek Gaber, Sunil Vadera, and Khaled Shaalan. A systematic literature review on phishing email detection using natural language processing techniques. *Cybersecurity Engineering*, pages 65703–65727, 10 2022. optional.

[Sha23] SharkStriker. Top 10 most common types of cyber attacks, 2023. Accessed: 2025-04-11.

[SpC22] Chanti Surya prakasam and T. Chithralekha. A literature review on classification of phishing attacks. *International Journal of Advanced Technology and Engineering Exploration*, 9:446–476, 04 2022.

[Sub24] Subhajournal. Phishing Emails – kaggle dataset, 2024.

[TM21] Lizhen Tang and Qusay H. Mahmoud. A survey of machine learning-based solutions for phishing website detection. *Machine Learning and Knowledge Extraction*, 3(3):672–694, 2021.

[VGG20] Priyanka Verma, Anjali Goyal, and Yogita Gigras. Email phishing: text classification using natural language processing. *Computer Science and Information Technologies*, 1:1–12, 05 2020.

[WKN+20] Wei Wei, Qiao Ke, Jakub Nowak, Marcin Korytkowski, Rafal Scherer, and Marcin Woźniak. Accurate and fast url phishing detector: A convolutional neural network approach. *Computer Networks*, 178:107275, 04 2020.

[Wor23] World Economic Forum. The global risks report 2023: 18th edition, 2023. Accessed: 2025-04-11.

[XHRC11] Guang Xiang, Jason Hong, Carolyn P. Rose, and Lorrie Cranor. Cantina+: A feature-rich machine learning framework for detecting phishing web sites. *ACM Trans. Inf. Syst. Secur.*, 14(2), September 2011.

[ZHC07] Yue Zhang, Jason I. Hong, and Lorrie F. Cranor. Cantina: a content-based approach to detecting phishing web sites. In *Proceedings of the 16th International Conference on World Wide Web*, WWW '07, page 639–648, New York, NY, USA, 2007. Association for Computing Machinery.

[ZO17] Mouad Zouina and Benaceur Outtaj. A novel lightweight url phishing detection system using svm and similarity index. *Human-centric Computing and Information Sciences*, 7(1):17, 2017.

List of Abbreviations

AI Artificial Intelligence. 2, 4, 19, 29, 34, 37, 41, 57

API Application Programming Interface. 33, 34, 36–38, 40, 42–47, 56

BERT Bidirectional Encoder Representations from Transformers. 12, 17, 18, 29, 36

CNN Convolutional Neural Networks. 16–18, 21

DL Deep Learning. 15, 17, 21, 24, 28, 32, 52

DT Decision Tree. 28, 52

JSON JavaScript Object Notation. 34, 46

JWT JSON Web Token. 38, 47

LLM Large Language Model. 12, 23

LR Logistic Regression. 28, 52

LSTM Long Short-Term Memory. 2, 16, 18, 21, 24–26, 52–55

ML Machine Learning. 2, 11–13, 17, 18, 21, 27, 28, 30, 32, 52, 55

NB Naive Bayes. 14, 28

NLP Natural Language Processing. 2, 11–14, 21, 23, 25, 36

ORM Object Relational Mapper. 34, 36

REST Representational State Transfer. 34, 36, 47

RF Random Forests. 13, 14, 21, 28, 52

RNN Recurrent Neural Networks. 16, 18

SVM Support Vector Machine. 12, 14, 21, 52

TF-IDF Term Frequency-Inverse Document Frequency. 2, 11, 19, 28

URL Uniform Resource Locator. 5, 12, 15, 19–23, 27, 29, 36, 40, 43, 45, 46, 56